Space-efficient iteration over an approximate subtrajectory clustering Pareto front

Ivor van der Hoog, Lara Ost, Eva Rotenberg, Daniel Rutschmann

arxiv:2503.14115; CC BY 4.0

Editorial scope. The PSC algorithm approximates an entire length/cardinality Pareto front through balanced-tree prefix and suffix cluster generation, dynamically maintained free-space reachability and sparse row generation with range trees; its source-specific complexity proof gives O(z log^4 n) time and O(n log n) space. Only PSC theorem counted. Full native source and explicitly used source assets retained under arxiv own CCBY4; author experiments are original context and no rerun or certification claimed.

Allowed standard prerequisites (editorial).

- Discrete Frechet free-space reachability, range trees and standard subtrajectory clustering primitives. Author full descriptions, algorithms and bibliography retained, including Appendix improved BBGLL routine and sparse row generation.

1 Introduction

Trajectories describe the movement of objects. The moving objects are modeled as a sequence of locations. Trajectory data often comes from GPS samples, which are being generated on an incredible scale through cars, cellphones, and other trackers. We focus on subtrajectory clustering where clusters are sets of

subtrajectories. The goal is to create few clusters and to ensure that subtrajectories within a cluster have low pairwise discrete Fréchet distance. This algorithmic problem has been studied extensively [3, 10, 12, 14, 15, 17, 18, 21, 23, 24, 22, 26, 29, 32] in both theoretical and applied settings. For surveys, see [9] and [30].

Subtrajectory clustering. Comparing whole trajectories gives little information about shared structures. Consider two trajectories T_1 and T_2 that represent individuals who drive the same route to work, but live in different locations. For almost all distance metrics, the distance between T_1 and T_2 is large because their endpoints are (relatively) far apart. However, T_1 and T_2 share similar subtrajectories. Subtrajectory clustering allows us to model a wide range of behaviours: from detecting commuting patterns, to flocking behaviour, to congestion areas [14]. In the special case where trajectories correspond to traffic on a known road network, trajectories can be mapped to the road network. In many cases, the underlying road network might not be known (or it may be too dense to represent). In such cases, it is interesting to construct a sparse representation of the underlying road network based on the trajectories. The latter algorithmic problem is called map construction, and it has also been studied extensively [4, 8, 5, 16, 19, 27, 28]. We note that subtrajectory clustering algorithms may be used for map construction: if each cluster is assigned one centre, then the collection of centres can be used to construct an underlying road network [11, 12].

Related work. BBGLL [14] introduce the following problem. Rather than clustering $\mathcal{T}$, their goal is to compute a single cluster. Given $\mathcal{T}$, a Δ-cluster is any pair $(P, \mathcal{P})$ where:

- P is a subtrajectory of a trajectory in $\mathcal{T}$,
- $\mathcal{P}$ is a set of subtrajectories, where for all $\mathcal{T}[a, b], \mathcal{T}[c, d] \in \mathcal{P}$, $[a, b] \cap [c, d] = \emptyset$,
- and, for all $P' \in \mathcal{P}$, the Fréchet distance between P and P' is at most Δ.

See Figure 1. They define two functions that each output a single Δ-cluster $(P, \mathcal{P})$:

- **SC(**max, ℓ, Δ, $\mathcal{T}$**)** requires that $|P| = \ell$ and maximises $|\mathcal{P}|$.
- **SC(**m, max, Δ, $\mathcal{T}$**)** requires that $|\mathcal{P}| = m$ and maximises $|P|$.

If $|P| = \ell$ and $|\mathcal{P}| = m$, their algorithms use $O(n^2 + nm\ell)$ time and $O(n\ell)$ space. AFMNPT [3] propose a clustering framework. They define a clustering C as any set of clusters. They propose a scoring function with constants (c_1, c_2, c_3) that respectively weigh: the number of clusters, the radius of each cluster, and the number of uncovered vertices. Their implement a heuristic algorithm that uses this scoring function. BBDFJSSSW [11] do map construction. They compute a set of Δ-clusters C for various Δ. For each $(P, \mathcal{P}) \in C$, they use P as a road on the map they are constructing. Thus, it may be argued that BBDFJSSSW [11] do subtrajectory clustering. As a preprocessing step, they implement **SC(**max, ℓ, Δ, $\mathcal{T}$**)**. By trying various choices for ℓ and Δ, they compute a collection S of 'candidate' clusters. On a high level, they greedily select a cluster $(P, \mathcal{P}) \in S$, add it to the clustering, and remove all subtrajectories in $\mathcal{P}$ from all remaining clusters in S. For a cluster $(Q, \mathcal{Q}) \in S$, this may split a $Q' \in \mathcal{Q}$ into two subtrajectories. BBGHSSSSSW [12] continue this line of work with improved implementations. ABCD [6] do not require that for all $\mathcal{T}[a, b], \mathcal{T}[c, d] \in \mathcal{P}$, $[a, b] \cap [c, d] = \emptyset$. They do require that the final clustering C has no uncovered vertices, and that for each cluster $(P, \mathcal{P})$, $|P| \leq \ell$ for some given ℓ. For any Δ, denote by k_Δ the minimum-sized clustering that meets their requirements. Their randomised polynomial-time algorithm computes a clustering of $O(\Delta)$-clusters of size $\tilde{O}(k_\Delta \ell)$. BCD [10] give polynomial-time clustering algorithm of $O(\Delta)$-clusters of size $\tilde{O}(k_\Delta)$. Conradi and Driemel [18] implement an $\tilde{O}(\ell^2 n^4 + k_\Delta \ell n^4)$-time algorithm that computes a set of $O(\Delta)$-clusters of size $O(k_\Delta \log n)$.

Contribution. Our contribution is threefold. First, in Section 5, we describe an efficient implementation of **SC(**max, ℓ, Δ, $\mathcal{T}$**)** and **SC(**m, max, Δ, $\mathcal{T}$**)**. We compare our implementation to all existing single-core implementations. In Section 5 we propose that **SC(**max, ℓ, Δ, $\mathcal{T}$**)** produces high-quality clusters across all clustering metrics. We create a greedy clustering algorithm that

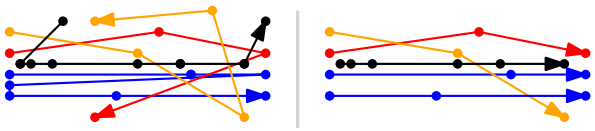


Figure 1: (left) Four trajectories. (right) Five subtrajectories $\mathcal{P}$. Depending on the choice of $P \in \mathcal{P}$, we may get five different clusters $(P, \mathcal{P})$.

fixes (ℓ, Δ) and iteratively adds the solution $(P, \mathcal{P}) =$ **SC(**max**,** ℓ**,** Δ**,** $\mathcal{T}$**)** to the clustering (removing all vertices in $\mathcal{P}$ from $\mathcal{T}$). An analogous approach for **SC(**m**,** max**,** Δ**,** $\mathcal{T}$**)** gives two clustering algorithms. In Section 6 we note that the functions **SC(**max**,** ℓ**,** Δ**,** $\mathcal{T}$**)** and **SC(**m**,** max**,** Δ**,** $\mathcal{T}$**)** search a highly restricted solution space, having fixed either the minimum length of the centre of a cluster or the minimum cardinality. We observe that these functions always output a cluster on the Pareto front of some bivariate function $\theta(m, \ell)$. Since points on this Pareto front give high-quality clusters, we argue that it may be beneficial to approximate all vertices of this Pareto front directly – sacrificing some cluster quality for cluster flexibility. We create a new algorithm **PSC(**Δ**,** $\mathcal{T}$**)** that computes in $O(n^2 \log^4 n)$ time and $O(n)$ space a set of clusters that is a 2-approximation of the Pareto font of $\theta(m, \ell)$. We show that using **PSC(**Δ**,** $\mathcal{T}$**)** as a subroutine for greedy clustering gives even better results. In Section 7 we do experimental analysis. We compare our clustering algorithms to AFMNPT [3] and BBGHSSSSSW [12]. We adapt the algorithm in [12] so that it terminates immediately after computing the subtrajectory clustering. We do not compare to Conradi and Driemel [18] as their setting is so different that it produces incomparable clusters. We compare the algorithms based on their overall run time and space usage. We evaluate the clustering quality along various metrics such as the number of clusters, the average cardinality of each cluster, and the Fréchet distance between subtrajectories in a cluster. Finally, we consider the scoring function by AFMNPT [3]. Our repository can be found here [1].

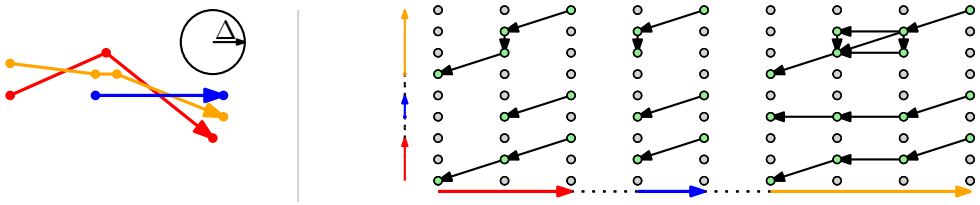


Figure 2: A set of trajectories $\mathcal{T}$ and the matrix $M_\Delta(\mathcal{T},\mathcal{T})$ as a graph.

2 Preliminaries

A trajectory T in $\mathbb{R}^2$ is an ordered sequence of vertices. The algorithmic input is a set $\mathcal{T}$ of trajectories where $n = \sum_{T\in\mathcal{T}} |T|$. We denote by $\mathcal{X}$ the set of all vertices in $T \in \mathcal{T}$. We view any trajectory T with n_T vertices as a map from $[n_T]$ to $\mathbb{R}^2$ where $T(i)$ is the i'th vertex on T. For brevity, we refer to $\mathcal{T}$ as a single trajectory obtained by concatenating all trajectories in $\mathcal{T}$ arbitrarily. A boundary edge in $\mathcal{T}$ is any edge in $\mathcal{T}$ between the endpoints of two consecutive trajectories. A subtrajectory $\mathcal{T}[a,b]$ of $\mathcal{T}$ is defined by $a, b \in [n]$ with $a \leq b$, where we require that the subcurve from $\mathcal{T}(a)$ to $\mathcal{T}(b)$ contains no boundary edge.

Fréchet distance. We first define discrete walks for pair of trajectories $P = (p_1, \ldots, p_x)$ and $T = (t_1, \ldots, t_y)$ in $\mathbb{R}^2$. We say that an ordered sequence F of points in $[x] \times [y]$ is a discrete walk if consecutive pair $(i,j),(s,l) \in F$ have $s \in \{i, i-1\}$ and $l \in \{j, j-1\}$. The cost of a discrete walk F is the maximum distance $d(P(i),Q(j))$ over $(i,j) \in F$. The discrete Fréchet distance is the minimum-cost discrete walk from (x,y) to $(1,1)$:

$$\mathcal{D}_F(P,Q) := \min_F \text{cost}(F) = \min_F \max_{(i,j)\in F} d(P(i),Q(j)).$$

Free-space matrix. Given $\mathcal{T}$ and a value Δ, the Free-space matrix $M_\Delta(\mathcal{T},\mathcal{T})$ is an $n \times n$ 01-matrix where for all $(i,j) \in [n] \times [n]$ the matrix has a zero at position i, j if $d(\mathcal{T}(i),\mathcal{T}(j)) \leq \Delta$. We use matrix notation where (i,j) denotes row i and column j. In the plane, $(0,0)$ denotes the top left corner. We also use $M_\Delta(\mathcal{T},\mathcal{T})$ to

denote the digraph where the vertices are all $(i, j) \in [n] \times [n]$ and there exists an edge from (i, j) to (s, k) whenever (see Figure 2):

- $s \in \{i, i-1\}$ and $k \in \{j, j-1\}$, and
- $M_\Delta(\mathcal{T}, \mathcal{T})[i, j] = M_\Delta(\mathcal{T}, \mathcal{T})[s, k] = 0$.

Alt and Godeau [7] show that for any two subtrajectories $\mathcal{T}[a, b]$ and $\mathcal{T}[c, d]$: $\mathcal{D}_F(T[a, b], T[c, d]) \leq \Delta$ if and only if there exists a directed path from (b, d) to (a, c) in this graph.

Definition 1. Given Δ and $\mathcal{T}$, we say that a vertex (b, x) in $M_\Delta(\mathcal{T}, \mathcal{T})$ can reach row a if there exists an integer $x' \leq x$ such that there is a directed path from (b, x) to (a, x'). We denote by $\mathit{row}(b, x)$ the minimum integer a' such that (b, x) can reach row a'.

Definition 2 (Figure 1)**.** A cluster is any pair $(P, \mathcal{P})$ where P is a subtrajectory and $\mathcal{P}$ is a set subtrajectories with $P \in \mathcal{P}$. We require that for all $\mathcal{T}[a, b], \mathcal{T}[c, d] \in \mathcal{T}$, $[a, b] \cap [c, d] = \emptyset$. We define its length $|P|$, cardinality $|\mathcal{P}|$, and coverage $\mathit{cov}(P, \mathcal{P}) = \bigcup_{S \in \mathcal{P}} \mathrm{S}$.

Note that we follow [3, 11, 12, 14] and consider only disjoint clusterings, meaning $\forall (P, \mathcal{P}') \in C$, $\forall \mathcal{T}[a, b], \mathcal{T}[c, d] \in \mathcal{P}$, the intervals $[a, b]$ and $[c, d]$ are disjoint. For brevity, we say that a Δ-cluster is any cluster $(P, \mathcal{P})$ where $\forall P' \in \mathcal{P} : \mathcal{D}_F(P, P') \leq \Delta$.

Definition 3. A clustering C is any set of clusters. Its coverage is $\mathit{cov}(C) := \bigcup_{(P,\mathcal{P}) \in C} \mathit{cov}(P, \mathcal{P})$.

AFMNPT [3] propose a cost function to determine the quality of a clustering C. We distinguish between k-centre and k-means clustering:

Definition 4. Given constants (c_1, c_2, c_3) , $\mathit{Score}(C)$ is a sum that weights three terms:

1. The number of clusters in C: $c_1|C|$.
2. The chosen cost function for the clustering. I.e., $c_2 \max_{(P,\mathcal{P}) \in C} \mathrm{cen}(P, \mathcal{P}) = c_2 \max_{(P,\mathcal{P}) \in C} \max_{P' \in \mathcal{P}} \mathcal{D}_F(P, P')$ when doing k-centre clustering, or $c_2 \sum_{(P,\mathcal{P}) \in C} \mathrm{mean}(P, \mathcal{P}) = c_2 \sum_{(P,\mathcal{P}) \in C} \sum_{P' \in \mathcal{P}} \mathcal{D}_F(P, P')$ when doing k-means clustering.

3. The fraction of uncovered points: $c_3 \frac{|\mathcal{X} \setminus \mathcal{cov}(C)|}{|\mathcal{X}|}$.

We also consider unweighted metrics: the maximal and average Fréchet distance between subtrajectories in a cluster, and the maximal and average cardinality of the clusters.

3 Data

We use five real-world data sets (see Table 1). For some of these data sets we include a subsampled version that only retains a $\frac{1}{c}$ fraction of the trajectories.

- The first three data sets are Athens-small, Chicago and Berlin, consisting of GPS samples from car traffic across these cities. These data sets are widely used benchmark tests for road map construction [4, 11, 33, 25].
- The fourth data set is the Drifter data set that was used for subtrajectory clustering by Driemel and Conradi [18]. It consists of GPS data from ocean drifters.
- The fifth data set is the UnID data set, which is the smallest data set used in [31].

Unfortunately, we cannot use any of the data sets in [3]. Most of these are no longer available. Those that are available are far too large. It is unknown how [3] preprocessed these.

Synthetic data. All competitor algorithms do not terminate on the larger real-world data sets, even when the time limit is 24 hours. Thus, we primarily use these sets to compare algorithmic efficiency. Our qualitative analysis uses the subsampled data sets and synthetic data. We generate data by creating a random domain X. Then, for some parameter c, we randomly select c% of the domain and invoke an approximate TSP solver. We add the result as a trajectory. This results in a collection of trajectories with many similar subtrajectories. Synthetic-A has a random domain of 200 vertices. Synthetic-B has a random domain of 100 vertices. For both of these, we consider $c \in \{50, 90, 95\}$.

Name	**Real world**	**# of trajectories**	**# of vertices**	**Subsampled**
Athens-small	yes	128	2840	no
Chicago-4	yes	222	29344	$\frac{1}{4}$'th
Chicago	yes	888	118360	no
Berlin-10	yes	2717	19130	$\frac{1}{10}$'th
Berlin	yes	27188	192223	no
Drifter	yes	2167	1931692	no
UniD	yes	362	214077	no
Synthetic-A	no	50	5000 - 9500	no
Synthetic-B	no	100	5000 - 9500	no

Table 1: Our data sets, their sizes, and whether we subsampled a fraction of their trajectories.

4 Introducing and solving SC(m, ℓ, Δ, $\mathcal{T}$)

BBGLL [14] introduce the problem **SC(m, ℓ, Δ, $\mathcal{T}$)**. The input are integers m, ℓ, Δ and a set of trajectories $\mathcal{T}$. The goal is to find a Δ-cluster $(P, \mathcal{P})$ where $|\mathcal{P}| \geq m$ and $|P| = \ell$. The problem statement may be slightly altered by maximising m or ℓ instead. This yields the problems **SC(**max**,** ℓ**,** Δ**,** $\mathcal{T}$**)** and **SC(**m**,** max**,** Δ**,** $\mathcal{T}$**)**. There exist three single-core implementations of these algorithms (the MOVETK library [2], map-construct [11] and its improvement: map-construct-rtrees [12] a version of the map-construct algorithm that uses r-trees). [2] uses $O(n^2)$ space and $O(n^2 + nm\ell)$ time. Both [11, 12] use $O(n\ell)$ space and $O(n^2 + nm\ell)$ time, and [12] offers running time improvements over [12]. We note that the latter implement a semi-continuous Fréchet distance variant instead of the discrete Fréchet distance. On the experimental input, switching between these metrics does not change the algorithmic output. We do not compare to the existing GPU-based implementation [23].

Contribution. We sketch how we optimise the BBGLL algorithm in Appendix A. Considers two integers a, b with $\ell = b - a$ (Figure 3). It alternates between `StepFirst` which increments a, `StepSecond` which increments b, and `Query` that given $(\Delta, \ell = b-a, m)$ aims to find a Δ-cluster $(P, \mathcal{P})$ with $P = \mathcal{T}[a, b]$ and $|\mathcal{P}| = m$. We apply two ideas:

1. **Windowing.** By [14], it suffices to keep only $M_\Delta(\mathcal{T}[a, b], \mathcal{T})$ in memory. Furthermore, one may store only the $z' \in O(n\ell)$ zeroes in this sub-matrix. This is implemented in [11, 12]. By

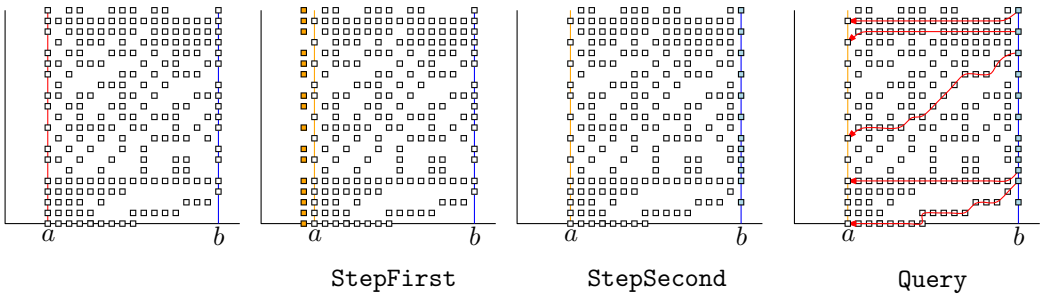


Figure 3: An overview of the algorithm in [14]. We illustrate zeroes in $M_\Delta(\mathcal{T}, \mathcal{T})$ with a square.

cleverly abandoning memory, our `StepFirst` deletes a row in constant time. The `StepSecond` function is then the bottleneck, as it adds a new row.

2. **Row generation.** In [14], the `StepSecond` and `Query` functions iterate over all n cells in the bottommost row of $M_\Delta(\mathcal{T}, \mathcal{T})$. By applying a range searching data structure, we instead only iterate over all zeroes in this row.

We note that map-construct-rtree [12] uses r-trees for row generation (we refer to the journal version [13]). However, we believe that the size of their r-tree implementation dominates the time and space used by their algorithm in our experiments. In Appendix A, we describe how to use a light-weight range tree to implement this principle.

Experiments. We compare the implementations across our data sets for a variety of choices of Δ and m. We use the implementations of **SC**(m, max, Δ, $\mathcal{T}$) so that our experiments have to fix fewer parameters. Figures 4 and 5 shows the time and space usage across a subset of our experiments. The data clearly shows that our implementation is significantly more efficient than the previous single-core implementations. It is often more than a factor 1000 more efficient in terms of runtime and memory usage. We do not provide a more extensive analysis of these results since this is not our main contribution.

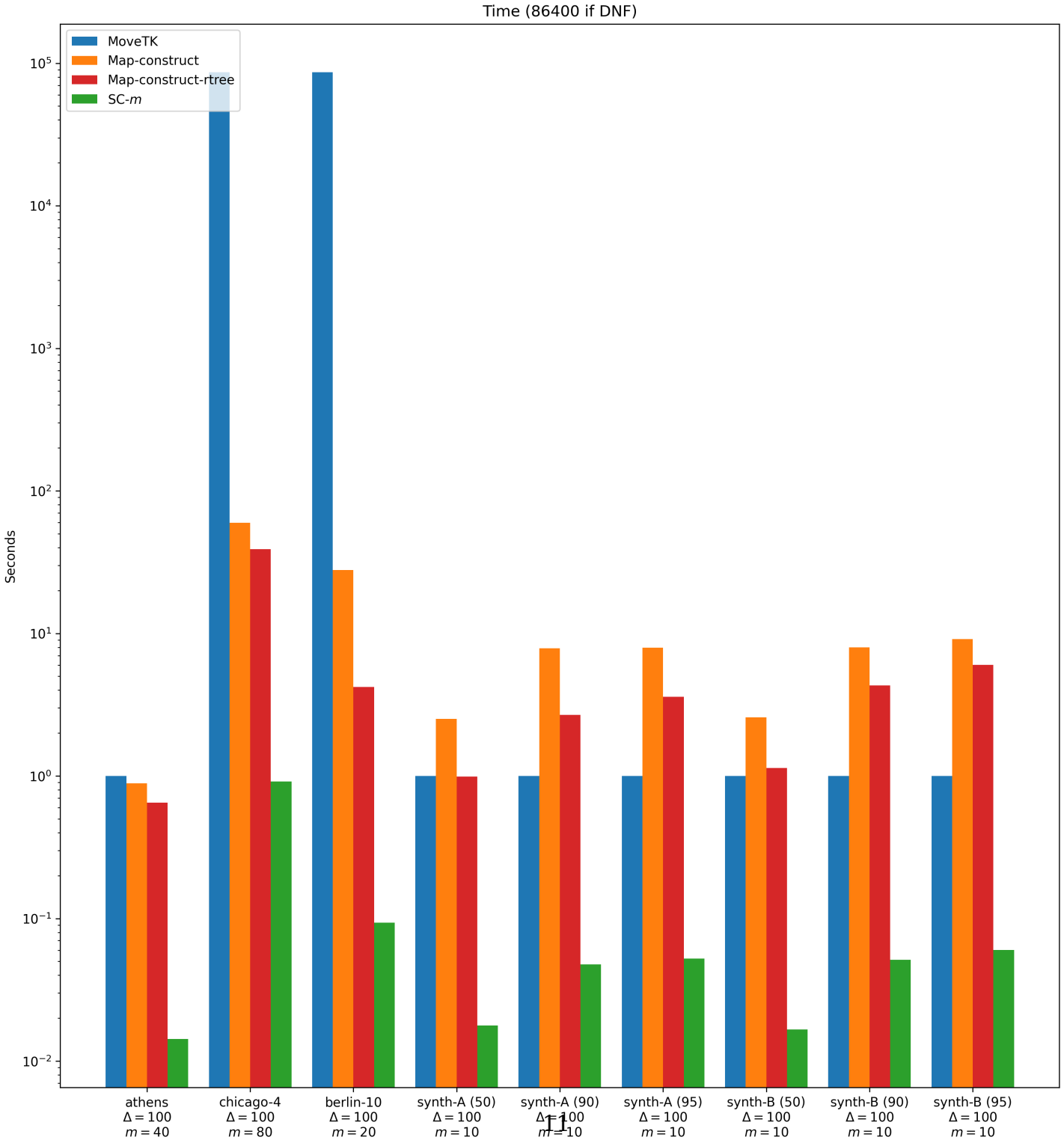


Figure 4: ■ BBGLL implementation comparison ■ Running time ■ Logarithmic scaling

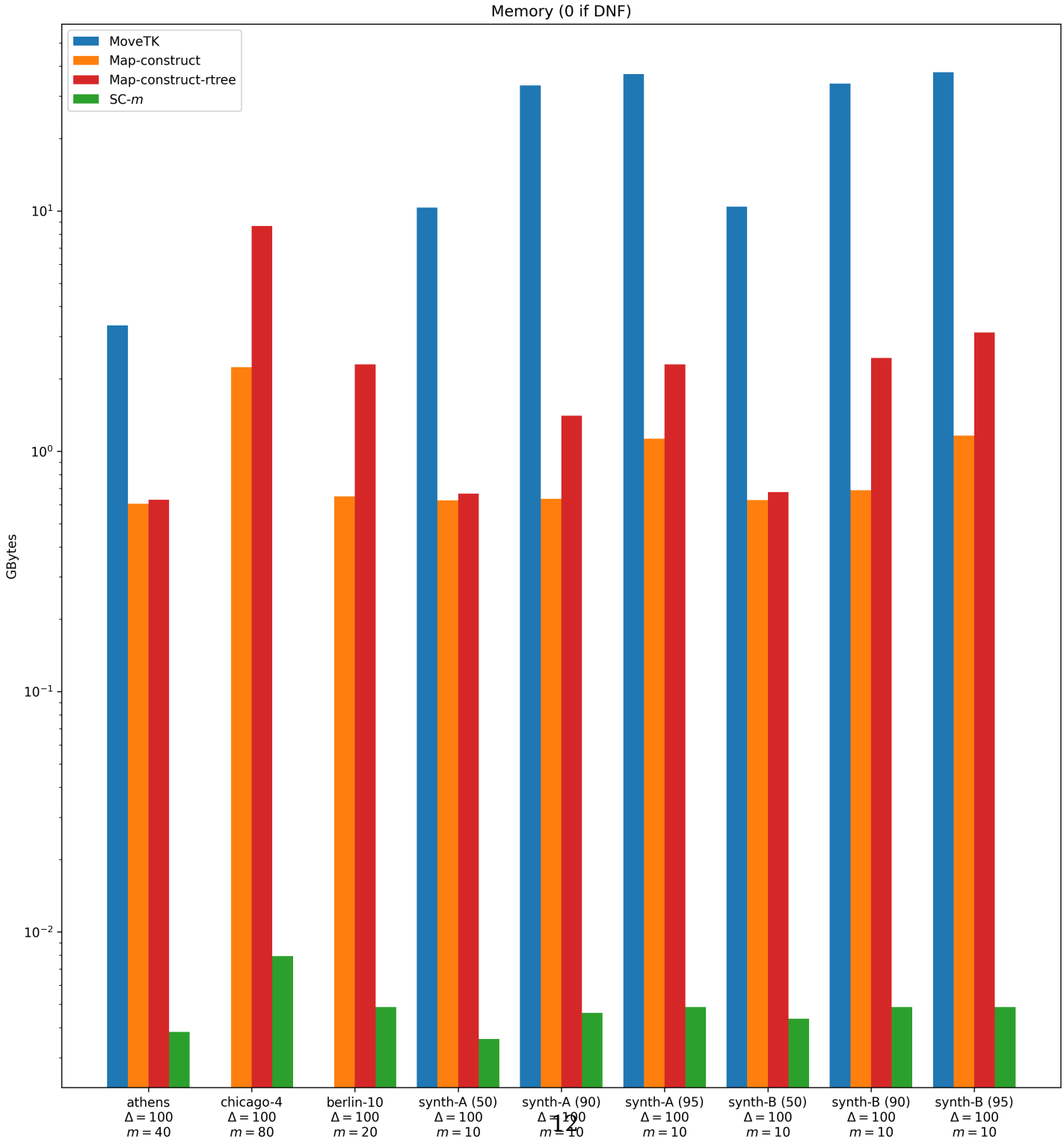


Figure 5: ■ BBGLL implementation comparison ■ Memory usage ■ Logarithmic scaling

5 Greedy clustering using SC(max, ℓ, Δ, $\mathcal{T}$)

Our primary contribution is that we propose to use **SC(**max**,** ℓ**,** Δ**,** $\mathcal{T}$**)** in a greedy clustering algorithm. On a high level, our clustering algorithm does the following:

- We first choose the scoring function and constants (see Definition 4).
- Our input is the set of trajectories $\mathcal{T}$, some Δ, some ℓ, and an empty clustering C.
- We obtain a cluster $(P, \mathcal{P})$ = **SC(**max**,** ℓ**,** Δ**,** $\mathcal{T}$**)**, and add it to C.
- We then remove all subtrajectories in $\mathcal{P}$ from $\mathcal{T}$, and recurse.
- We continue this process until the addition of $(P, \mathcal{P})$ to C no longer increases $\texttt{Score}(C)$.

The implementation details depend on whether we are doing k-centre or k-median clustering.

5.1 k-centre clustering.

Given are (c_1, c_2, c_3) and a set of trajectories $\mathcal{T}$. We choose some integer ℓ. The k-centre clustering scoring function weighs the value: $\max_{(P,\mathcal{P})\in C} \text{cen}(P, \mathcal{P}) = \max_{(P,\mathcal{P})\in C} \max_{P'\in\mathcal{P}} \mathcal{D}_F(P, P')$. If C is a set of Δ-clusters (we assume there exists a cluster in C whose radius is Δ) then:

$$\texttt{Score}(C) = c_1 \cdot |C| + c_2 \cdot \Delta + c_3 \cdot \frac{|\mathcal{X} \backslash \texttt{cov}(C)|}{|\mathcal{X}|}.$$

Let C be a set of Δ-clusters and $(P, \mathcal{P})$ = **SC(**max, ℓ, Δ, $\mathcal{T} - \texttt{cov}(C)$**)**. Per construction of our algorithm, $\texttt{cov}(P, \mathcal{P}) \cap \texttt{cov}(C) = \emptyset$ and so the contribution of $(P, \mathcal{P})$ to $\texttt{Score}(C \cup \{(P, \mathcal{P})\})$ equals $c_1 - c_3 \cdot \frac{|\texttt{cov}(P,\mathcal{P})|}{|\mathcal{X}|}$. We obtain $|\texttt{cov}(P, \mathcal{P})|$ using constant additional time throughout our algorithm. This leads to the following algorithm:

Clustering 1 (SC-$\ell$$((c_1, c_2, c_3), \mathcal{T})$ under k-centre clustering**).** We run Algorithm 1 (c_1, c_2, c_3) for exponentially increasing $\Delta \in (2, 4, \ldots)$ where the subroutine f is **SC**(max, ℓ, Δ, $\mathcal{T}$). We then output the maximum-score clustering C over all runs.

Algorithm 1 `k_centre`(`vector` (c_1, c_2, c_3), `value` Δ, `trajectory` $\mathcal{T}$, `subroutine` f)

$C \leftarrow \emptyset$, $T \leftarrow \mathcal{T}$.
while $|T| > 0$ **do**
  $(P, \mathcal{P}) \leftarrow$ the result of applying f to (Δ, T)
  if `Score`$(C \cup \{(P, \mathcal{P})\}) >$ `Score`(C) **then**
    C.add($(P, \mathcal{P})$).
    $T \leftarrow T - \mathcal{P}$.
  else
    Break.
  end if
end while
return C

We may also fix m instead of ℓ. Or, formally:

Clustering 2 (SC-m((c_1, c_2, c_3), $\mathcal{T}$) under k-centre clustering **).** We run Algorithm 1 (c_1, c_2, c_3) for exponentially increasing $\Delta \in (2, 4, \ldots)$ where the subroutine f is **SC(**m, max, Δ, $\mathcal{T}$**)**. We then output the maximum-score clustering C over all runs.

5.2 k-means clustering.

Our algorithm for k-means clustering is less straightforward. To get good performance we must iterate over different choices of Δ across the algorithm. Consider a clustering C and $(P, \mathcal{P}) =$ **SC(**max**,** ℓ**,** Δ**,** $\mathcal{T} -$ `cov`(C)**)** for some Δ. Under k-centre clustering, the more points a Δ-cluster covers, the better its scoring contribution. For k-means clustering this is no longer true as the contribution of $(P, \mathcal{P})$ to `Score`$(C \cup \{(P, \mathcal{P})\})$ equals:

$$\text{Contribution of } (P, \mathcal{P}) \text{ to } \texttt{Score}(C) = c_1 - c_3 \cdot \frac{|\texttt{cov}(P, \mathcal{P})|}{|\mathcal{X}|} + \sum_{P' \in \mathcal{P}} c_2 \cdot \mathcal{D}_F(P, P').$$

We abuse the fact that subtrajectories share no vertices to simplify the above expression to:

$$\text{Contribution of } (P, \mathcal{P}) \text{ to } \texttt{Score}(C) = c_1 + \sum_{P' \in \mathcal{P}} \left(c_2 \cdot \mathcal{D}_F(P, P') - c_3 \frac{|P'|}{|\mathcal{X}|} \right).$$

Consider a set of 'candidate' clusters S. To select the best cluster from S, we use the same greedy set cover technique as [3]. We

assign to every cluster a evaluation value. This is the ratio between how much it reduces the score, and how much it increases the score. I.e.,

$$\texttt{evaluation}(P, \mathcal{P}) = \frac{\sum_{P' \in \mathcal{P}} c_3 \frac{|P'|}{|\mathcal{X}|}}{c_1 + \sum_{P' \in \mathcal{P}} c_2 \cdot \mathcal{D}_F(P, P')}$$

We then select the cluster with the best evaluation among all $(P, \mathcal{P}) \in S$.

Clustering 3 (SC-ℓ((c_1, c_2, c_3), $\mathcal{T}$) under k-means clustering**).** We run Algorithm 2 where the subroutine f is **SC**(max, ℓ, Δ, $\mathcal{T}$).

Clustering 4 (SC-m((c_1, c_2, c_3), $\mathcal{T}$) under k-means clustering). We run Algorithm 2 where the subroutine f is **SC**(m, max, Δ, $\mathcal{T}$).

Algorithm 2 `k_means`(`vector` (c_1, c_2, c_3), `trajectory` $\mathcal{T}$, `subroutine` f)

$C \leftarrow \emptyset$, $T \leftarrow \mathcal{T}$.
while $|T| > 0$ **do**
  $(P, \mathcal{P}) \leftarrow$ null
  for $\Delta \in (2, 4, \ldots)$ **do**
    $(Q, \mathcal{Q}) \leftarrow$ the result of applying f to (Δ, $\mathcal{T}$)
    if `evaluation`$(Q, \mathcal{Q}) >$ `evaluation`$(P, \mathcal{P})$ **then**
      $(P, \mathcal{P}) \leftarrow (Q, \mathcal{Q})$.
    end if
  end for
  if `Score`$(C \cup \{(P, \mathcal{P})\}) >$ `Score`(C)) **then**
    C.Add($(P, \mathcal{P})$).
    $T \leftarrow T - \mathcal{P}$.
  else
    Break.
  end if
end while
return C

6 A new algorithm as a subroutine

Our previous clusterings 1-4 fix a parameter ℓ (or m) and subsequently only produce clusters $(P, \mathcal{P})$ of maximal cardinality with $|P| = \ell$ (or of maximum length with $|\mathcal{P}| = m$). This approach has three clear downsides:

1. To find a suitable ℓ (or m) you have to train the algorithm on a smaller data set.
2. Using multiple values of ℓ (or m) to broaden the search space is computationally expensive.
3. Even if we fix constantly many choices for ℓ and m, the corresponding algorithm has a heavily restricted solution space which affects the quality of the output.

To alleviate these downsides, we design a new algorithm based on the following observation:

Definition 5. For fixed $(\Delta, \mathcal{T})$, we define a bivariate Boolean function $\theta(m, \ell)$ where $\theta(m, \ell)$ is 1 if and only if there exists a Δ-cluster $(P, \mathcal{P})$ of $\mathcal{T}$ with $m = |\mathcal{P}|$ and $\ell = |P|$.

The functions **SC(**max**,** ℓ**,** Δ**,** $\mathcal{T}$**)** and **SC(**m**,** max**,** Δ**,** $\mathcal{T}$**)** always output a Δ-cluster $(P, \mathcal{P})$ where $\theta(|P|, |\mathcal{P}|)$ lies on the Pareto Front of $\theta(\cdot\,, \cdot)$. If clusters on the Pareto front of $\theta(\cdot\,, \cdot)$ are good candidates to include in a clustering, then it may be worthwhile to directly compute an approximation of this Pareto front as a candidate set:

Definition 6. Given $(\Delta, \mathcal{T})$, a set of Δ-clusters S is a 2-approximate Pareto front if for every Δ-cluster $(P, \mathcal{P})$ there exists a $(Q, \mathcal{Q}) \in S$ with: $|\mathcal{P}| \leq |\mathcal{Q}|$ and $|P| \leq 2|Q|$.

Our new algorithm, **PSC(**Δ**,** $\mathcal{T}$**)**, iterates over a 2-approximate Pareto front. Storing this front explicitly uses $O(n^2 \log n)$ space. Instead, we generate these clusters on the fly.

High-level description. Let $B(\mathcal{T})$ be a balanced binary tree over $\mathcal{T}$. Let $S(\Delta, \mathcal{T})$ start out as empty. Our algorithm **PSC**$(\Delta, \mathcal{T})$ (defined by Algorithm 3) does the following:

- For each inner node in B, denote by $\mathcal{T}[a,b]$ the corresponding subtrajectory of $\mathcal{T}$.
- For every prefix or suffix P of $\mathcal{T}[a,b]$, let $(P,\mathcal{P})$ be a Δ-cluster maximising $|\mathcal{P}|$.
- We add $(P,\mathcal{P})$ to $S(\Delta,\mathcal{T})$.

Computing all prefix clusters for $\mathcal{T}[a,b]$. Given a trajectory $\mathcal{T}[a,b]$ and our current set $S(\Delta,\mathcal{T})$, we simultaneously compute for all prefixes P of $\mathcal{T}[a,b]$ a maximum-cardinality cluster $(P,\mathcal{P})$ and add this cluster to $S(\Delta,\mathcal{T})$. We define an analogue of Definition 1:

Definition 7. Consider row a in $M_\Delta(\mathcal{T},\mathcal{T})$. For $(x,y)\in M_\Delta(\mathcal{T},\mathcal{T})$ let $\texttt{col}_a(x,y)$ be the maximum column $d\le y$ such that there exists a directed path from (x,y) to (a,d).

Given $\mathcal{T}[a,b]$ we initialize our algorithm by storing integers a and $c\leftarrow a$. We increment c and maintain for all $y\in[n]$ the value $\texttt{col}_a(c,y)$. This uses $O(n)$ space, as does storing row c. Given this data structure, we alternate between a `StepSecond` and `Query` subroutine: `StepSecond`: We increment c by 1. For each $y\in[n]$, we

first check whether $d(\mathcal{T}(c),\mathcal{T}(y))>\Delta$. If it is then (c,y) is an isolated vertex in the graph $M_\Delta(\mathcal{T},\mathcal{T})$. Thus, there exists no directed path from (c,y) to any vertex in row a and we set $\texttt{col}_a(c,y)=-\infty$. Otherwise, $\texttt{col}_a(c,y)=\max\{\texttt{col}_a(\alpha,\beta)\mid\alpha\in\{c,c-1\},\beta\in\{y,y-1\}\}$ which we compute in $O(1)$ time. `Query`: For a prefix $P=\mathcal{T}[a,c]$ of $\mathcal{T}[a,b]$ we compute a maximum-cardinality Δ-cluster $(P,\mathcal{P})$ using the values $\texttt{col}_a(c,y)$ (see Algorithm 4). Note that Algorithm 4 is essentially the same as the algorithm in [14] that we described in Section 5. Thus, it computes a maximum-cardinality cluster. The key difference is that we, through $\texttt{col}_a(b,j_2)$, avoid walking through the free-space matrix in $O(\ell)$ time to compute j_1. We denote by `Query'`$(T[c,b],\ \Delta,\ \mathcal{T},\ S(\Delta,\mathcal{T}))$ the function which computes for a suffix $P=\mathcal{T}[c,b]$ of $\mathcal{T}[a,b]$ a maximum-cardinality cluster $(P,\mathcal{P})$ in an analogous manner.

Lemma 1. The set $S(\Delta,\mathcal{T})$ is a 2-approximate Pareto front.

Proof. Consider any Δ-cluster $(P, \mathcal{P})$. Let $P = \mathcal{T}[a, b]$. Let $\mathcal{T}[i, k]$ be the minimal subtrajectory in our tree that has $\mathcal{T}[a, b]$ as a subtrajectory. Let the children of $\mathcal{T}[i, k]$ in our tree be $\mathcal{T}[i, j]$ and $\mathcal{T}[j+1, k]$. Then, $a \leq j < b$ by minimality of $\mathcal{T}[i, k]$. Hence, $\mathcal{T}[a, j]$ is a suffix of $\mathcal{T}[i, j]$, and $\mathcal{T}[j+1, b]$ is a prefix of $\mathcal{T}[j+1, k]$. There must exist a trajectory $P' \in \{\mathcal{T}[a, j], \mathcal{T}[j+1, b])\}$ which contains at least half as many vertices as $T[a, b]$. Since P' is either a prefix or suffix of a subtrajectory in our tree, we must have added some maximum-cardinality cluster $(P', \mathcal{P}')$ to $S(\Delta, \mathcal{T})$. Moreover, since P' is a subtrajectory of P, $|\mathcal{P}'| \geq |\mathcal{P}|$ (since any discrete walk from row b to row a is also a discrete walk in the Free-space matrix from row d to row c for $[c, d] \subseteq [a, b]$). □

Theorem 1. The algorithm PSC(Δ, $\mathcal{T}$) iterates over a 2-approximate Pareto front of $\mathcal{T}$ in $O(z \log^4 n)$ time using $O(n \log n$ space. Here, z denotes the number of zeroes in $M_\Delta(\mathcal{T}, \mathcal{T})$.

Proof. Consider the balanced binary tree $B(\mathcal{T})$. Each level of depth of $B(\mathcal{T})$ corresponds to a set of subtrajectories that together cover $\mathcal{T}$. For every such subtrajectory $\mathcal{T}[a, b]$, both `StepSecond` and `Query` get invoked for each row $c \in [a, b]$. Since the tree has depth $\log n$, it follows that we invoke these functions $O(\log n)$ times per row c of $M_\Delta(\mathcal{T}, \mathcal{T})$ (i.e. $O(n \log n)$ times in total). Naively, both the `StepSecond` and `Query` functions iterate over all columns in their given row c. However, just as in Section 5, we note that our functions skip over all (j_2, c) for which $d(\mathcal{T}(j_2), \mathcal{T}(c)) > \Delta$. It follows that we may use our row generation trick to spend $O(Z_c \log^3 n)$ time instead (Z_c denotes the number of zeroes in row c). Since each row c gets processed $O(\log n)$ times, this upper bounds our running time by $O(z \log^4 n)$. For space, we note that invoking `StepSecond` and `Query` for a row c need only rows c and $c-1$ in memory. So the total memory usage is $O(n)$. Explicitly storing $S(\Delta, \mathcal{T})$ takes $O(n^2 \log n)$ space. However, since we only iterate over $S(\Delta, \mathcal{T})$, the total space usage remains $O(n)$. The range tree therefore dominates the space with its $O(n \log n)$ space usage. □

6.1 Greedy clustering using PSC(Δ, $\mathcal{T}$)

We reconsider Algorithms 1 and 2 in Section 5 for k-centre and k-means clustering. As written, these algorithms invoke a sub-

routine f that returns a single cluster $(P, \mathcal{P})$. Then, the algorithm computes whether it wants to add $(P, \mathcal{P})$ to the current clustering. We can adapt these algorithms to this new function. For f, we invoke **PSC**$(\Delta, \mathcal{T})$. This function iterates over a set of clusters $(P, \mathcal{P})$. For each cluster $(P, \mathcal{P})$ that we encounter, we let either Algorithm 1 or 2 compute whether it wants to add $(P, \mathcal{P})$ to the current clustering.

7 Experiments

We propose three new greedy clustering algorithms for k-centre clustering:

1. SC-ℓ runs Algorithm 1, for exponentially scaling Δ, with $f \leftarrow$**SC(**max**,** ℓ**,** Δ**,** $\mathcal{T}$**)**,
2. SC-m runs Algorithm 1, for exponentially scaling Δ, with $f \leftarrow$ **SC(**m**,** max**,** Δ**,** $\mathcal{T}$**)**, and
3. PSC runs Algorithm 1, for exponentially scaling Δ, with $f \leftarrow$ **PSC**$(\Delta, \mathcal{T})$.

Our algorithms SC-ℓ and SC-m need to fix a parameter ℓ and m respectively. We choose $\ell \in \{4, 8, 16, 32, 64\}$ and $m \in \{2, 4, 8, 16\}$ and do a separate run for each choice. Thus, together with PSC, we run ten different algorithm configurations. We additionally compare to the subtrajectory clustering algorithm from [12], which is oblivious to the scoring function.

- `Map-construction` runs the algorithm in [12]. We terminate this map construction algorithm as soon as it has constructed a clustering of $\mathcal{T}$.

k-means clustering. For k-means clustering, we obtain three algorithms by running Algorithm 2 instead. Note that our clustering algorithms invoked Algorithm 1 across several runs for exponentially scaling Δ. Our clustering algorithms do not invoke Algorithm 2 in this manner. Instead, Algorithm 2 considers exponentially scaling Δ's internally. Our empirical analysis will show that this change has significant ramifications for the clustering's

performance on large input. We again obtain ten algorithm configurations. In addition, we run the map construction algorithm from [12] and the k-means subtrajectory clustering algorithms from [3]:

- `Envelope` runs the algorithm from [3] with the scoring parameters (c_1, c_2, c_3).

Experimental setup. We design different experiments to measure performance and the scoring function. For performance, we run fourteen experiments across the five real-world data sets. Seven experiments for k-centre clustering, and seven for k-means clustering. Each experiment ran three times, using a different vector (c_1, c_2, c_3). Each experiment runs 11 programs for k-centre clustering, or 12 for k-means clustering. If a program fails to terminate after 24 hours, we do not extract any output or memory usage. We restrict scoring measurements to the three smaller or subsampled real-world data sets. This is necessary to make `Map-construct` and `Envelope` terminate. We add six synthetic data sets. We only compare under k-means clustering since only our algorithms can optimise for k-centre clustering. The experiments were conducted on a machine with a 4.2GHz AMD Ryzen 9 7950X3D and 128GB memory. Over 90 runs hit the timeout. Running the total set of experiments sequentially takes over 100 days on this machine. The full results per experiment are organised in tables in Appendix C. In this section, we restrict our attention to only three plots.

Choosing (c_1, c_2, c_3). Recall that c_1 weighs the number of clusters in the clustering and that c_3 weighs the fraction of uncovered points. We follow the precedent set in [3] and choose $c_1 = 1$, and c_3 equal to the number of trajectories in $\mathcal{T}$. Finally, c_2 weighs the Fréchet distance across clusters. For each data set, we first find a choice for c_2 such the algorithms lead to a non-trivial clustering. We then also run the algorithms using the scoring vectors $(c_1, 10 \cdot c_2, c_3)$ and $(c_1, 0.1 \cdot c_2, c_3)$, for a total of three scoring vectors per data set.

Comparing quality. To compare solution quality, we measure the scoring function and other qualities such as: the total number of clusters in the clustering, the maximum Fréchet distance from a subtrajectory to its centre, and the average Fréchet distance across clusters. Our competitor algorithms frequently time out on the real-world data sets. Hence, the qualitative comparison also makes use of our synthetic data set. We use three different configurations of our synthetic set to create input of different size. We choose to compare only under k-means clustering for a more fair comparison to other algorithms. Indeed, the algorithm by [3] was designed for the k-means metric ([12] is scoring function oblivious). Consequently, comparing under k-centre would only give us an unfair advantage.

7.1 Results for k-means clustering

The bar plot in Figure 6 visualizes how the algorithms score. Figures 7 and 8 show, respectively, a logarithmic-scale plot of the time and space usage. These plots use the 'medium' choice for c_2 for each data set. Note that our choice of c_2 is very similar across the data sets in the figure. This is because Athens, Berlin and Chicago come from the same source, and thereby have the same scaling. We subsequently designed our synthetic data to match this scaling. Plots for the other scoring vector choices are contained in Appendix B.

Clustering quality. Figure 6 shows that `Map-construct` (which is oblivious to the scoring function) scores poorly on all inputs. One reasons for this is that it always outputs a large number of clusters. The `Envelope` algorithm, which was specifically designed for this scoring function, almost always obtains the best score. For any data set, for any scoring vector, we always find a choice for ℓ (or m) where `SC-`ℓ and `SC-`m obtain a comparable score to `Envelope`. The algorithm `PSC` scores best out of all newly proposed clustering algorithms. Across all data sets and scoring functions it frequently competes with, and sometimes even improves upon, the score of `Envelope`.

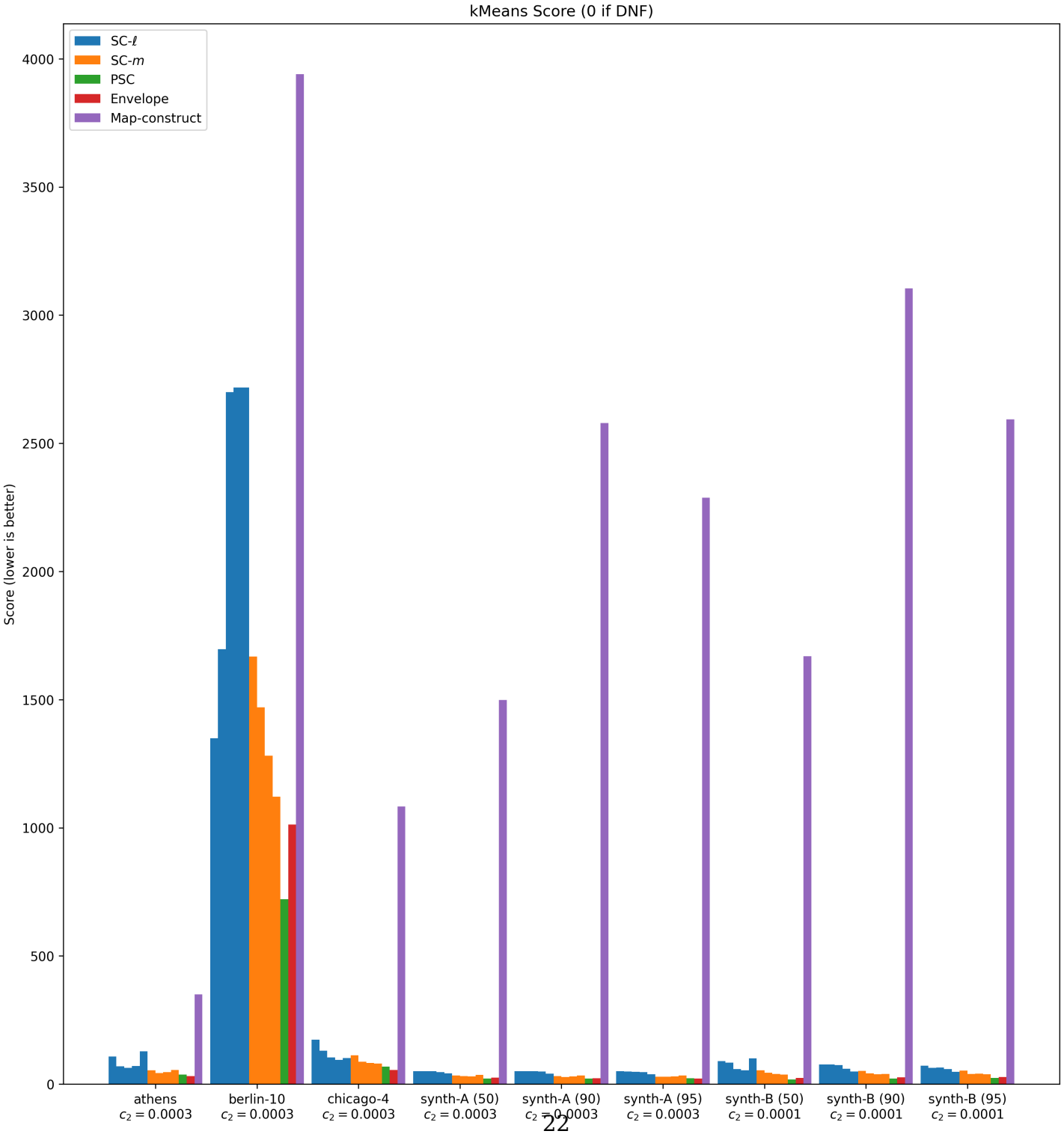


Figure 6: ■ k-means clustering ■ k-means score ■

Other quality metrics. We briefly note that the full data in the appendix also considers other quality metrics: the number of clusters and the maximum and average Fréchet distance within a cluster. Perhaps surprisingly, `Map-construct` does not perform better on the latter two metrics even though it outputs many clusters. `Envelope` scores worse on these metrics across nearly all data sets and scoring vectors. However, it stays within a competitive range.

Runtime comparison. Figure 7 shows the runtime performance in logarithmic scale. On the Berlin data set, no algorithm terminates in time. On Drifter and UniD, only `PSC` terminates within the time limit. In the figure, we consider the synthetic data sets their place. `Envelope` may produce high-quality clusterings, but takes significant time to do so. Despite using considerably less memory than `Map-construct`, it is consistently more than a factor two slower. Our algorithms `SC-`ℓ and `SC-`m can always find a choice of ℓ or m such that previous algorithms are orders of magnitude slower. Our `PSC` algorithm is always orders of magnitude faster than the existing clustering algorithms. This is especially true on larger data sets, which illustrates that our algorithms have better scaling. When we compare our own algorithms, we note that we can frequently find a choice for ℓ such that `SC-`ℓ is significantly faster than `SC-`m and `PSC`. However, when considering the runtime to quality ratio, `PSC` appears to be a clear winner.

Memory consumption. Figure 8 shows memory performance on a logarithmic scale. The figure excludes Drifter, UniD and Berlin for the same reason as when comparing running times. We record zero memory usage whenever an algorithm times out. We observe that `Map-construct` uses an order of magnitude more memory than its competitors. This is because it stores up to $O(n^2)$ candidate clusters in memory, together with the entire free-space matrix $M_\Delta(\mathcal{T}, \mathcal{T})$ for various choices of Δ. `Envelope` also has a large memory footprint and uses at least a factor 100 more memory on all of the inputs. When comparing our own implementations, we observe that `SC-`ℓ and `PSC` have a very small memory footprint. These algorithms use $O(n\ell)$ and $O(n)$ space. Since we fix ℓ to be a

small constant, both algorithms use near linear space. The space usage of SC-m is $O(n\ell)$ where ℓ is the length of the longest subtrajectory P such that there exists a Δ-cluster $(P, \mathcal{P})$ with $|\mathcal{P}| \geq m$. The data shows that for many choices of (m, Δ), this $O(n\ell)$ space is considerable. Yet, it is still considerably lower than pre-existing algorithms.

7.2 Results for k-centre clustering

When comparing score, runtime, or memory usage under k-centre clustering, we reach the same conclusions. The only difference is that the performance of SC-m becomes more competitive to SC-ℓ and PSC on large data sets. This is because **SC**(m, max, Δ, $\mathcal{T}$) uses much time and space whenever both the data set and Δ are large. Our k-means clustering meta-algorithm considers a large Δ each round, and invokes **SC**(m, max, Δ, $\mathcal{T}$) every round. Our k-centre clustering meta-algorithm has a separate run for each Δ. If Δ is large, it terminates quickly because it adds very high-cardinality clusters to the clustering. This speeds up all our implementations, but SC-m the most.

8 Conclusion

We proposed new subtrajectory clustering algorithms. Our first two approaches use the algorithms **SC**(max, ℓ, Δ, $\mathcal{T}$) and **SC**(m, max, Δ, $\mathcal{T}$) by BBGLL [14]. We use these algorithms as a subroutine in a greedy clustering algorithm to cluster a set of trajectories $\mathcal{T}$. Consider the Boolean function $\theta(\ell, m)$ that outputs `true` if there exists a Δ-cluster $(P, \mathcal{P})$ of $\mathcal{T}$ with $|P| = \ell$ and $|\mathcal{P}| = m$. We observe that these functions always output a point on the Pareto front of this function. We show a new algorithm **PSC**(Δ, $\mathcal{T}$) that, instead of returning a single cluster on this Pareto front, iterates over a 2-approximation of this Pareto front instead. Intuitively, this function iterates over a broader set of candidate clusters that have comparable quality to any output of **SC**(max, ℓ, Δ, $\mathcal{T}$) and **SC**(m, max, Δ, $\mathcal{T}$). We create a greedy clustering algorithm PSC using this algorithm as a subroutine. Our analysis shows that our new clustering algorithms significantly outperform previous algorithms in

running time and space usage. We primarily use the scoring function from [3] to measure clustering quality and observe that our algorithms give competitive scores. For other quality metrics, our algorithms often perform better than their competitors. We observe that PSC is the best algorithm when comparing the ratio between score and performance.

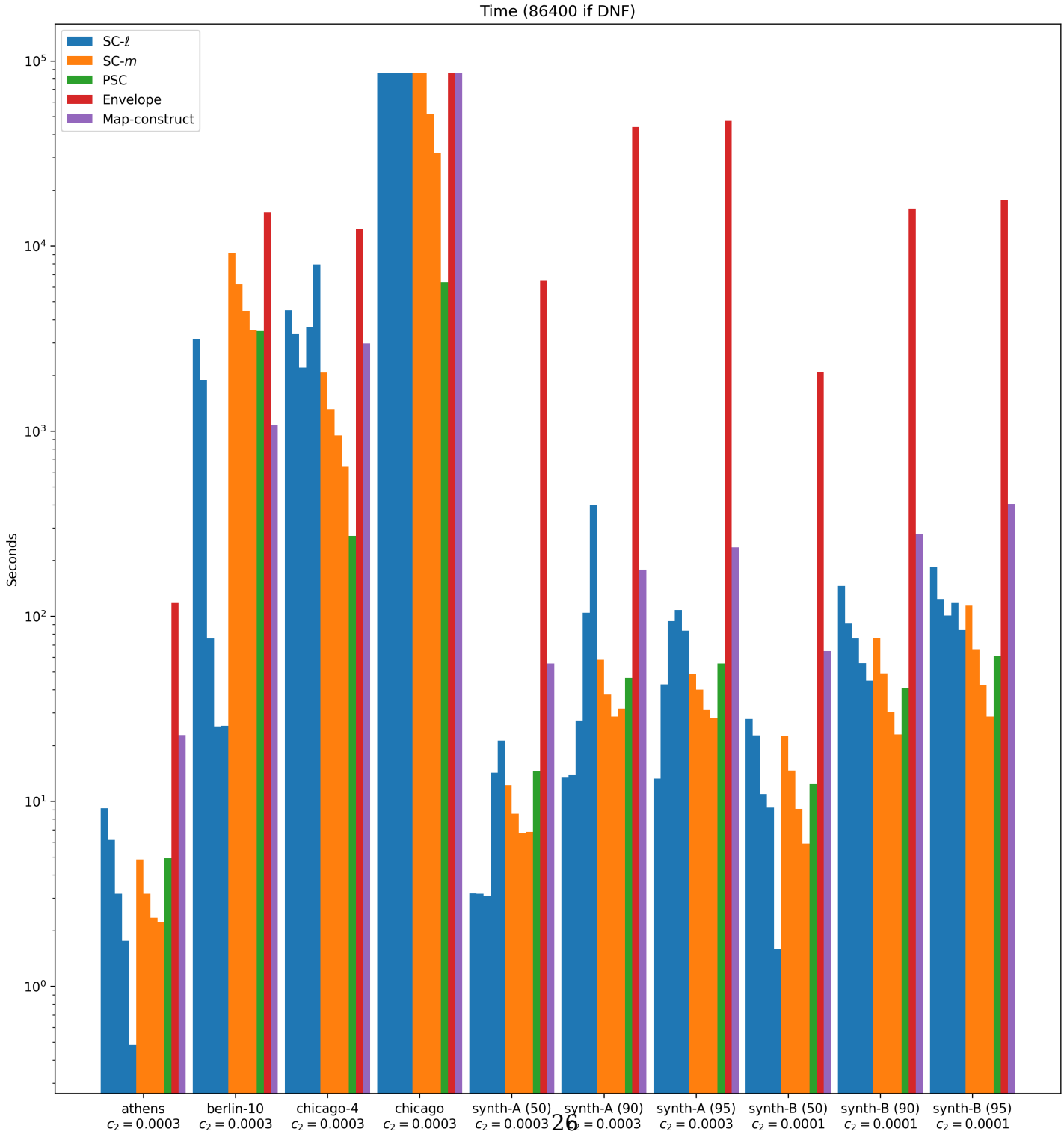


Figure 7: ■ k-means clustering ■ Running time ■ Logarithmic scaling

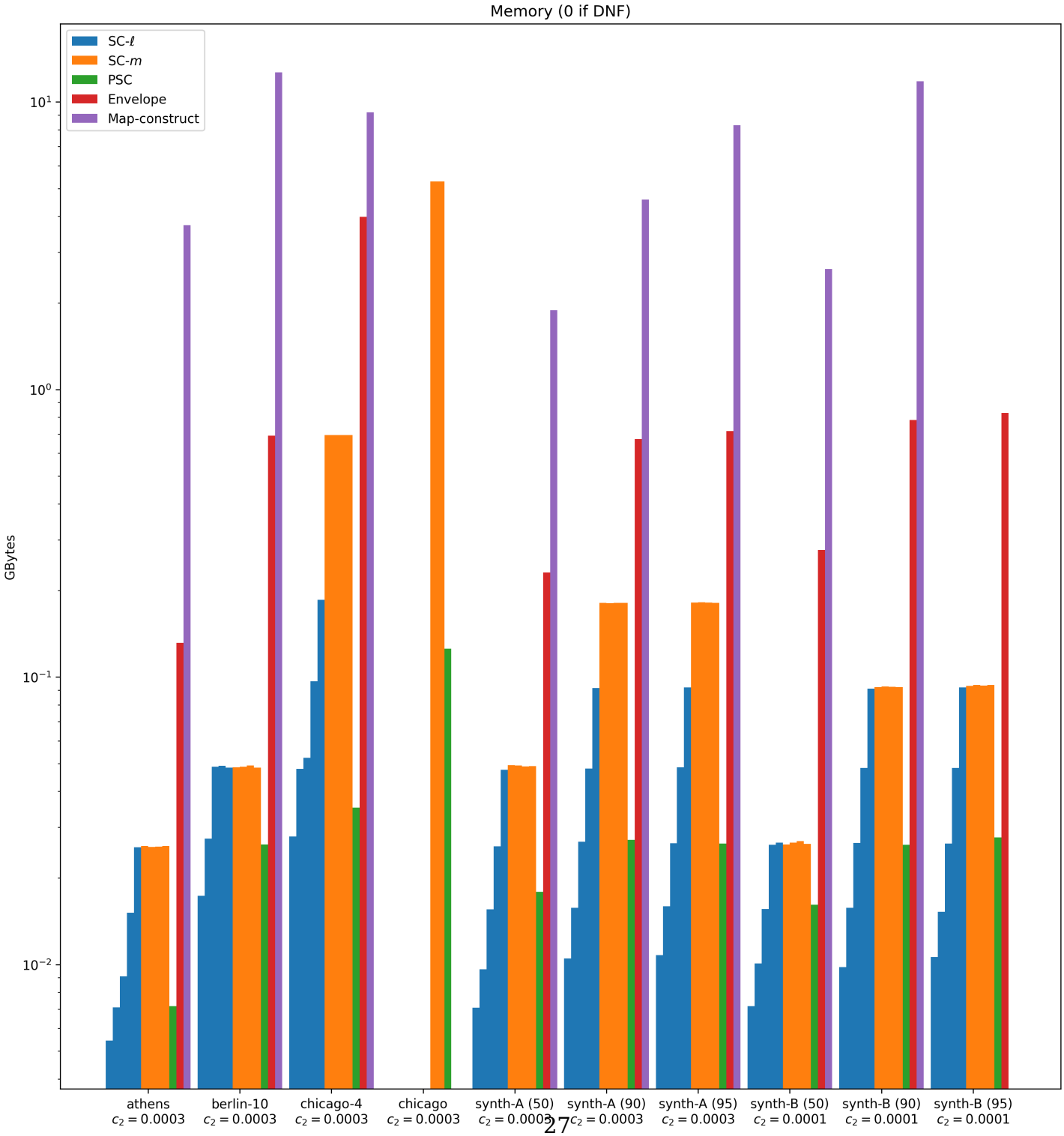


Figure 8: ■ k-means clustering ■ Memory usage ■ Logarithmic scaling
Whenever a run did not terminate, we display a memory usage of zero.

A Introducing and solving SC(m, ℓ, Δ, $\mathcal{T}$)

BBGLL [14] introduce the problem **SC(**m**,** ℓ**,** Δ**,** $\mathcal{T}$**)**. The input are integers m, ℓ, Δ and a set of trajectories $\mathcal{T}$. The goal is to find a Δ-cluster $(P, \mathcal{P})$ where $|\mathcal{P}| \geq m$ and $|P| = \ell$. The problem statement may be slightly altered by maximizing m or ℓ instead, giving the problems **SC(**max**,** ℓ**,** Δ**,** $\mathcal{T}$**)** and **SC(**m**,** max**,** Δ**,** $\mathcal{T}$**)**. They present a theoretical algorithm to solve **SC(**m**,** ℓ**,** Δ**,** $\mathcal{T}$**)** in $O(nm\ell)$ time and $O(n\ell)$ space. This algorithm can be adapted to solve the other two problem variants too. There exist two single-core implementations of this algorithm (the MOVETK library [2], and the one by [11, 12]). [2] uses $O(n^2)$ space and $O(n^2+nm\ell)$ time. We conjecture that [11, 12] use $O(n\ell)$ space and $O(n^2+nm\ell)$ time. There also exists a GPU-based implementation of this algorithm [23]. We do not consider this approach since we have no way to fairly compare single- and multi-core performance. We provide an optimised implementation of the BBGLL algorithm that significantly improves upon previous single-core implementations. We are unsure how many of our optimisations are included in [11, 12]. Theoretically, we slightly improve the asymptotic running time compared to [14], as we use $O(nm\ell+(n+z)\log^3 n)$ time instead of $O(nm\ell+n^2)$ time (where z denotes the number of zeroes in $M_\Delta(\mathcal{T},\mathcal{T})$).

The existing implementations. We explain the implementation of **SC(**m**,** ℓ**,** Δ**,** $\mathcal{T}$**)** by [2]. As a preprocessing step, store the matrix $M_\Delta(\mathcal{T},\mathcal{T})$ where each cell (i,j) stores $\texttt{row}(i,j)$ (Definition 1). After preprocessing, the algorithm has at all times in memory an input trajectory $T \in \mathcal{T}$, and two integers $\alpha \leq \beta \leq |T|$ where $\beta = \alpha + \ell - 1$. We use the notation $P = T[\alpha,\beta] = \mathcal{T}[a,b]$. The algorithm alternates between three functions:
`StepFirst`: increment α (and thus a).
`StepSecond`: if $\beta < |T|$, increment β (and thus b). Else let T' succeed T in $\mathcal{T}$, set $\alpha \leftarrow 1$, $\beta \leftarrow \ell$, $T \leftarrow T'$, $a \leftarrow b+1$ and $b \leftarrow a+\ell$.
`Query`: given $P = \mathcal{T}[a,b]$, greedily compute an m-cardinality Δ-cluster $(P,\mathcal{P})$ as described in Algorithm 5.

Algorithm analysis. Computing $M_\Delta(\mathcal{T},\mathcal{T})$ and the associated integers in each cell uses $O(n^2)$ space and can be computed in $O(n^2)$ time using a sweepline. The `StepFirst` and `StepSecond` func-

tions take constant time. The if-statement can be true at most m times and then takes $O(\ell)$ time. The else statement may occur at most $O(n)$ times before we get a point that is out of bounds, therefore the total query time is at most $O(n + m\ell)$. BBGLL [14] prove that one can solve SC(m, ℓ, Δ, $\mathcal{T}$) by invoking `StepFirst`, `StepSecond` and `Query` $O(n)$ times for a total runtime of $O(n^2 + nm\ell)$.

A.1 Two performance improvement techniques

We discuss two techniques that improve the performance of our implementation.

Windowing. The implementation in [2] computes the matrix $M_\Delta(\mathcal{T}, \mathcal{T})$ in $O(n^2)$ time and space. The original publication BBGLL [14] already describes how to reduce this space usage to $O(n\ell)$ by expanding the `StepFirst` and `StepSecond` functions, storing only the $\ell \times n$ submatrix/subgraph $M_\Delta(P = \mathcal{T}[a, b], \mathcal{T})$. We store only the $z' \in O(n\ell)$ non-isolated vertices in this subgraph. What remains is to dynamically maintain $M_\Delta(P = \mathcal{T}[a, b], \mathcal{T})$:
`StepFirst`: Before incrementing a, remove the top row from $M_\Delta(P, \mathcal{T})$ from memory.
`StepSecond`: After incrementing b, we compute the new row in the graph $M_\Delta(P, \mathcal{T})$ in $O(n)$ time. Iterate over all n columns from low to high. For each point (b, j) there are at most 3 cells with an in-edge to (b, j). Moreover, if $M_\Delta(b, j) = 0$ then $\texttt{row}(b, j)$ is the minimum of $\texttt{row}(b', j')$ for $b' \in \{b, b-1\}$ and $j' \in \{j, j-1\}$, which we access in $O(1)$ time.

Row generation. `StepSecond` and `Query` now share a bottleneck that we will call row generation. `StepSecond` iterates over all columns in a new row. `Query` (through the else-statement) iterates over $O(n)$ columns in the bottom-most row. Both subroutines can be adjusted to have an output-sensitive running time instead through the following structure:

- Construct a binary tree over $\mathcal{T}$ by recursively splitting the concatenated trajectory into two roughly equal-sized children. Store for each node all vertices in the corresponding subtrajectory in a range tree [20].

- Given (Δ, T', q) with $\Delta \in \mathbb{R}$, T' a subtrajectory corresponding to a node in our tree, and a query point q, the corresponding range tree can return the furthest point along T' that is within distance Δ of q in $O(\log^2 n)$ total time.

We use this data structure to speed up both functions.
`StepSecond`: For a given integer b, we need to compute for all $j \in [n]$ the value $\texttt{row}(b, j)$. Note that $\texttt{row}(b, j) < b$ only if $M_\Delta(T, \mathcal{T})[b, j] = 0$ (i.e., $\mathcal{T}(j)$ is within distance Δ of $T(b)$). We generate all such (b, j) from high to low: Let us have added (b, j) to the output. The trajectory $\mathcal{T}[1, j-1]$ consists of $O(\log n)$ subtrees of our binary tree over $\mathcal{T}$. We query our each subtree with $q = T(b)$ get the furthest point along the subtree within distance Δ of $T(b)$ in $O(\log^3 n)$ time. Thus, we add k points to our graph using $O(k \log^3 n)$ total time.
`Query`: If in the row generation we carefully record all (b, j) with $\texttt{row}(b, j) < a$, then we can avoid Case 2. This improves the run time of one `Query` call to $O(m\ell)$.
The `Query` function gets invoked $O(n)$ times. The `StepSecond` functions collectively use $O(n+z)$ range queries. Overall, this reduces the total running time to $O(nm\ell + (n+z)\log^3 n)$.

A.2 Experimental evaluation

Our experiments fix a data set $\mathcal{T}$ and some ℓ and Δ. We then run **SC**(m, max, Δ, $\mathcal{T}$). Consequently, we have to fix one fewer parameter compared to running **SC**(m, ℓ, Δ, $\mathcal{T}$). For each data set, we find some (m_0, Δ_0) that yields a reasonable output, then then consider $m \in \{m_0/2, m_0, 2m_0\}$ and $\Delta \in \{\Delta_0/2, \Delta_0, 2\Delta_0\}$. This way, we cover a wide range of parameters. There now exist four single-core implementations of the algorithm by BBGLL [14]:

1. `SC-`m refers to the C++ implementation found in this paper.
2. `Map-construct` ([11]) refers to the Java implementation in [11],
3. `Map-construct-rtree` refers to the Java implementation in [12], and
4. `Movetk` [2] refers to the C++ implementation in [2].

Data sets. We compare these algorithms using the Athens-small, Chicago-4, Berlin-10 and Synthetic data set. All other data sets are so large that Movetk runs out of memory before terminating. The raw data is given in three tables that follow this section. We discuss the results of these experiments through two bar plots that we show in the main body.

Algorithm 3 PSC(Δ, $\mathcal{T}$)

$S(\Delta, \mathcal{T}) \leftarrow \emptyset$.
$B(\mathcal{T}) \leftarrow$ balanced binary tree over $\mathcal{T}$.
for all $T[a, b] \in B(\mathcal{T})$ **do**
  for all $c \in [a, b]$ **do**
    `StepSecond`(Δ, $\mathcal{T}$, c).
    `Query`($T[a, c]$, Δ, $\mathcal{T}$, $S(\Delta, \mathcal{T})$).
  end for
  for all $c \in [a, b]$ in decreasing order **do**
    `StepSecond`(Δ, $\mathcal{T}$, c).
    `Query'`($T[c, b]$, Δ, $\mathcal{T}$, $S(\Delta, \mathcal{T})$).
  end for
end for
return $S(\Delta, \mathcal{T})$

Algorithm 4 Query($P = \mathcal{T}[a, c]$, Δ, $\mathcal{T}$, $S(\Delta, \mathcal{T})$)

$(c, j_2) \leftarrow M_\Delta(\mathcal{T}, \mathcal{T})[c, n]$, $\mathcal{P} \leftarrow \emptyset$
while $j_2 > 0$ **do**
  if $\texttt{col}_a(c, j_2) \neq -\infty$ **then**
    $j_1 \leftarrow \texttt{col}_a(b, j_2)$.
    Add $\mathcal{T}[j_1, j_2]$ to $\mathcal{P}$.
    $(c, j_2) \leftarrow (c, j_1 - 1)$.
  else
    $(c, j_2) \leftarrow (c, j_2 - 1)$.
  end if
end while
Add $(P, \mathcal{P})$ to $S(\Delta, \mathcal{T})$.

Algorithm 5 `Query`($P = \mathcal{T}[a, b]$, m, ℓ, Δ, $\mathcal{T}$) by [14]

$(b, j_2) \leftarrow (b, n)$ and $\mathcal{P} \leftarrow \emptyset$.
while $|\mathcal{P}| < m$ and $j_2 > 0$ **do**
  if $\texttt{row}(b, j_2) \leq a$ **then**
    Find the maximum j_1 such that there exists a path from (b, j_2) to (a, j_1).
    This takes $O(\ell)$ time by greedily stepping to the right-most neighbour (b', j') where,
    $\texttt{row}(b', j') \leq a$.
    Add $\mathcal{T}[j_1, j_2]$ to $\mathcal{P}$ and set $(b, j_2) \leftarrow (b, j_1 - 1)$.
  else
    $(b, j_2) \leftarrow (b, j_2 - 1)$.
  end if
end while
if $|\mathcal{P}| \geq m$ **then**
  return $(P, \mathcal{P})$
end if

Name	Seconds	GBytes	Δ	m
MoveTK	1.00	3.21	50	20
Map-construct	0.87	0.62	50	20
Map-construct-rtree	0.54	0.61	50	20
SC-m	0.01	0.00	50	20
MoveTK	1.00	3.21	50	40
Map-construct	0.78	0.61	50	40
Map-construct-rtree	0.52	0.61	50	40
SC-m	0.01	0.00	50	40
MoveTK	1.00	3.21	50	80
Map-construct	0.76	0.61	50	80
Map-construct-rtree	0.52	0.60	50	80
SC-m	0.01	0.00	50	80
MoveTK	1.00	3.34	100	20
Map-construct	0.92	0.62	100	20
Map-construct-rtree	0.66	0.62	100	20
SC-m	0.01	0.00	100	20
MoveTK	1.00	3.34	100	40
Map-construct	0.89	0.61	100	40
Map-construct-rtree	0.65	0.63	100	40
SC-m	0.01	0.00	100	40
MoveTK	1.00	3.34	100	80
Map-construct	0.85	0.63	100	80
Map-construct-rtree	0.66	0.62	100	80
SC-m	0.02	0.00	100	80
MoveTK	1.00	3.59	200	20
Map-construct	1.22	0.62	200	20
Map-construct-rtree	1.00	0.66	200	20
SC-m	0.02	0.00	200	20
MoveTK	1.00	3.59	200	40
Map-construct	1.12	0.62	200	40
Map-construct-rtree	0.99	0.66	200	40
SC-m	0.02	0.00	200	40
MoveTK	1.00	3.59	200	80
Map-construct	0.97	0.60	200	80
Map-construct-rtree	0.98	0.63	200	80
SC-m	0.03	0.00	200	80

Figure 9: ■ **SC**(m, max, Δ, $\mathcal{T}$) ■ Athens-small ■

Name	Seconds	GBytes	Δ	m
MoveTK	—	—	50	10
Map-construct	27.18	0.65	50	10
Map-construct-rtree	2.79	1.34	50	10
SC-m	0.05	0.00	50	10
MoveTK	—	—	50	20
Map-construct	24.69	0.62	50	20
Map-construct-rtree	2.75	1.32	50	20
SC-m	0.05	0.00	50	20
MoveTK	—	—	50	40
Map-construct	22.15	0.63	50	40
Map-construct-rtree	2.63	1.22	50	40
SC-m	0.06	0.00	50	40
MoveTK	—	—	100	10
Map-construct	27.12	0.66	100	10
Map-construct-rtree	4.03	2.32	100	10
SC-m	0.08	0.00	100	10
MoveTK	—	—	100	20
Map-construct	27.89	0.65	100	20
Map-construct-rtree	4.20	2.30	100	20
SC-m	0.09	0.00	100	20
MoveTK	—	—	100	40
Map-construct	25.81	0.66	100	40
Map-construct-rtree	4.22	2.27	100	40
SC-m	0.11	0.00	100	40
MoveTK	—	—	200	10
Map-construct	33.43	0.71	200	10
Map-construct-rtree	8.19	4.51	200	10
SC-m	0.18	0.01	200	10
MoveTK	—	—	200	20
Map-construct	31.32	1.16	200	20
Map-construct-rtree	7.76	4.50	200	20
SC-m	0.19	0.00	200	20
MoveTK	—	—	200	40
Map-construct	30.23	0.70	200	40
Map-construct-rtree	7.97	4.63	200	40
SC-m	0.21	0.01	200	40

Figure 10: ■ **SC**(m, max, Δ, $\mathcal{T}$) ■ Berlin-10 ■
`MoveTK` exceeds the 128GB memory limit in all nine runs.

Name	Seconds	GBytes	Δ	m
MoveTK	—	—	50	40
Map-construct	64.03	1.17	50	40
Map-construct-rtree	25.41	4.45	50	40
SC-m	0.48	0.01	50	40
MoveTK	—	—	50	80
Map-construct	54.49	0.80	50	80
Map-construct-rtree	24.37	2.27	50	80
SC-m	0.48	0.01	50	80
MoveTK	—	—	50	160
Map-construct	42.75	0.73	50	160
Map-construct-rtree	24.83	2.23	50	160
SC-m	0.58	0.01	50	160
MoveTK	—	—	100	40
Map-construct	73.22	2.17	100	40
Map-construct-rtree	38.80	9.12	100	40
SC-m	0.82	0.01	100	40
MoveTK	—	—	100	80
Map-construct	59.63	2.24	100	80
Map-construct-rtree	39.01	8.66	100	80
SC-m	0.92	0.01	100	80
MoveTK	—	—	100	160
Map-construct	51.59	0.72	100	160
Map-construct-rtree	38.71	8.44	100	160
SC-m	1.06	0.01	100	160
MoveTK	—	—	200	40
Map-construct	87.29	4.52	200	40
Map-construct-rtree	68.88	10.87	200	40
SC-m	1.80	0.02	200	40
MoveTK	—	—	200	80
Map-construct	80.58	4.25	200	80
Map-construct-rtree	70.14	10.64	200	80
SC-m	2.11	0.02	200	80
MoveTK	—	—	200	160
Map-construct	59.23	2.17	200	160
Map-construct-rtree	70.61	10.53	200	160
SC-m	2.53	0.02	200	160

Figure 11: ■ **SC**(m, max, Δ, $\mathcal{T}$) ■ Chicago-4 ■
`MoveTK` exceeds the 128GB memory limit in all nine runs.

B Bar plots for more variables

For each data set we run all algorithms using three choices for the scoring vector (c_1, c_2, c_3). In particular, we find for each data set some suitable triple (c_1, c_2, c_3). Then, we also consider the vectors $(c_1, 0.1c_2, c_3)$ and $(c_1, 10c_2, c_3)$. We call these three vectors the 'small', 'medium' and 'large' scoring vectors. Note that the choice of scoring vector matters even when we only compare algorithmic performance. This is because all involved algorithms make different algorithmic choices depending on the scoring parameters that they receive.

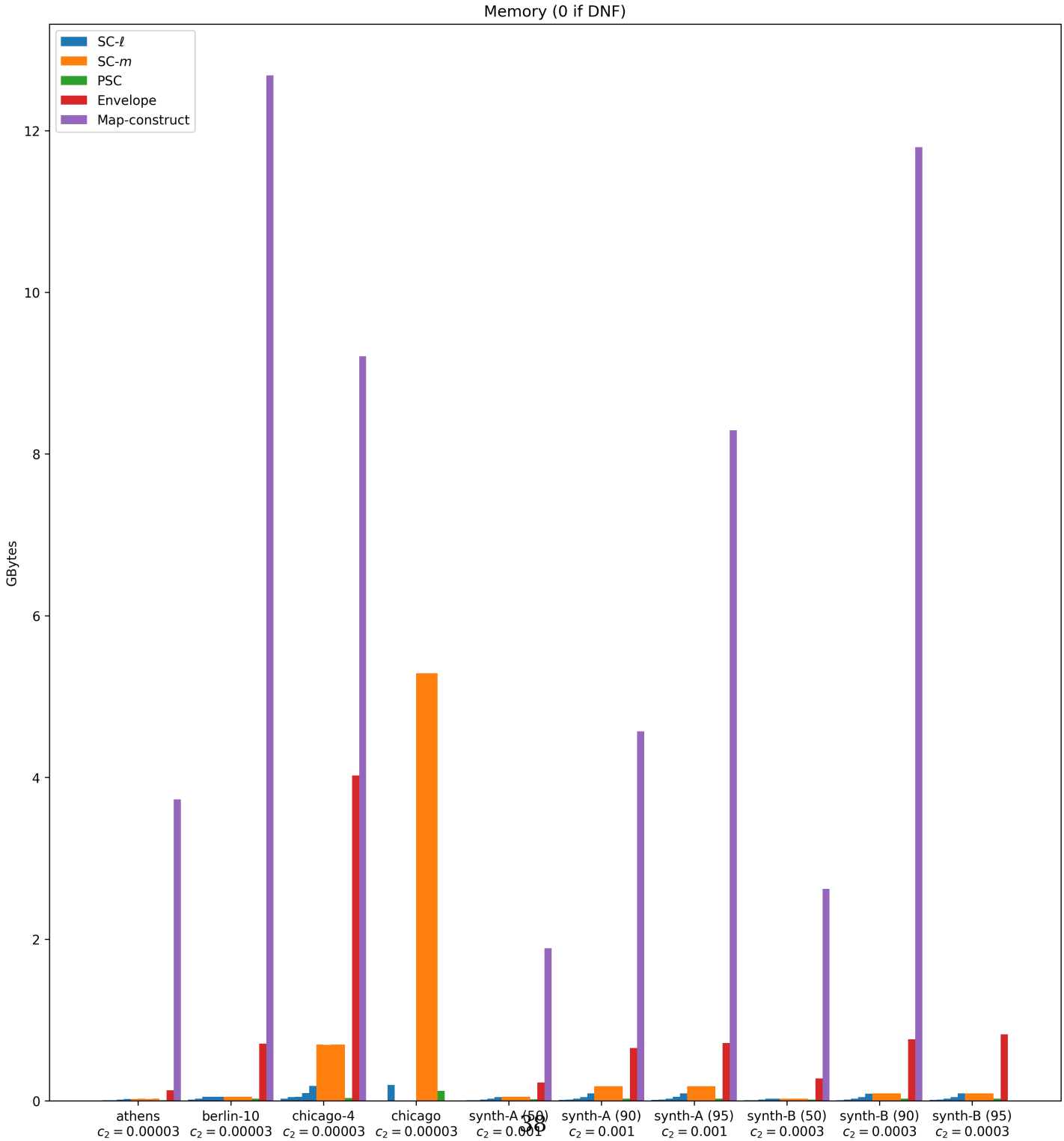


Figure 12: • k-means clustering • Memory usage • Small c_2 •

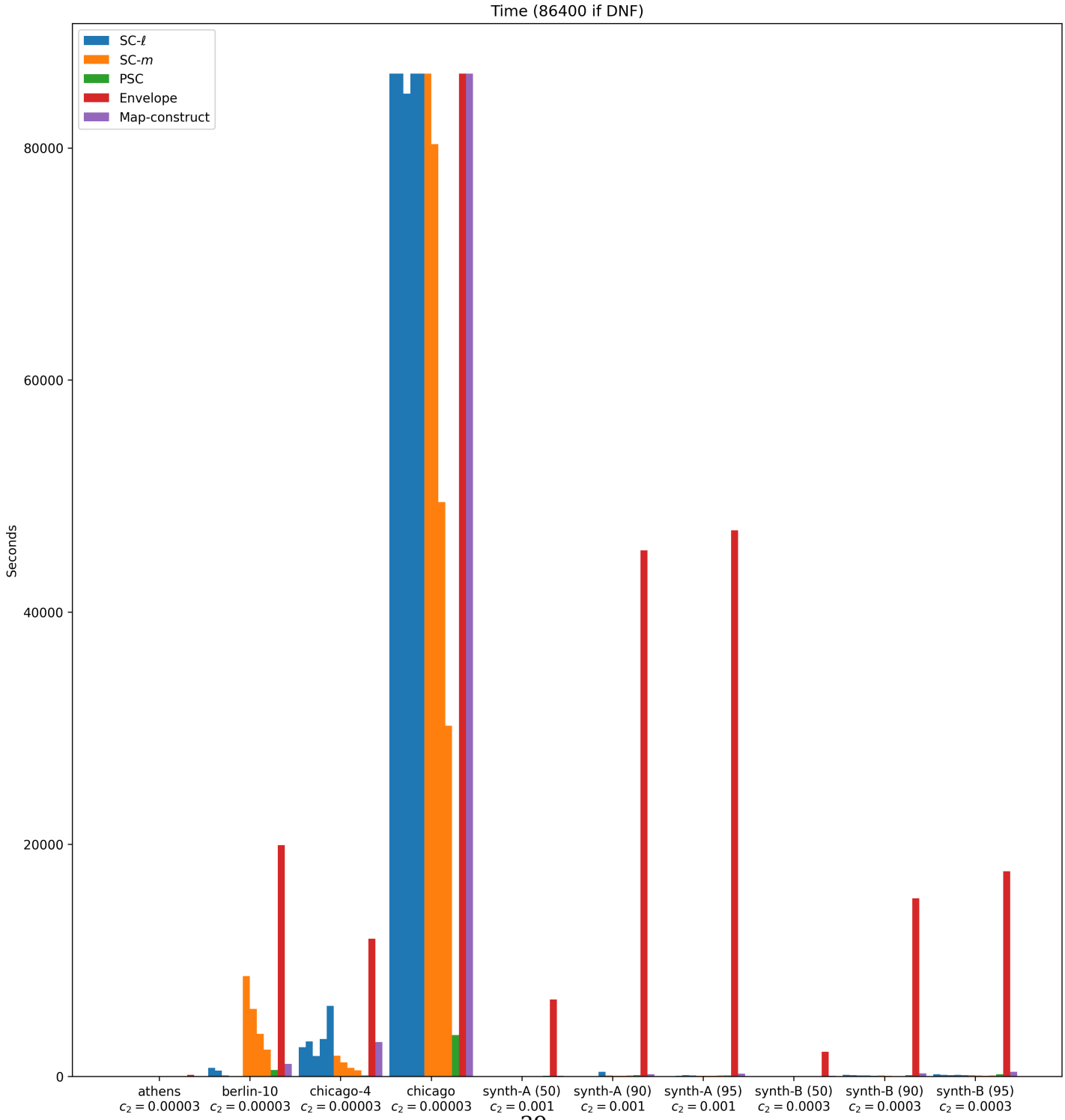


Figure 13: • k-means clustering • Running time • Small c_2 •

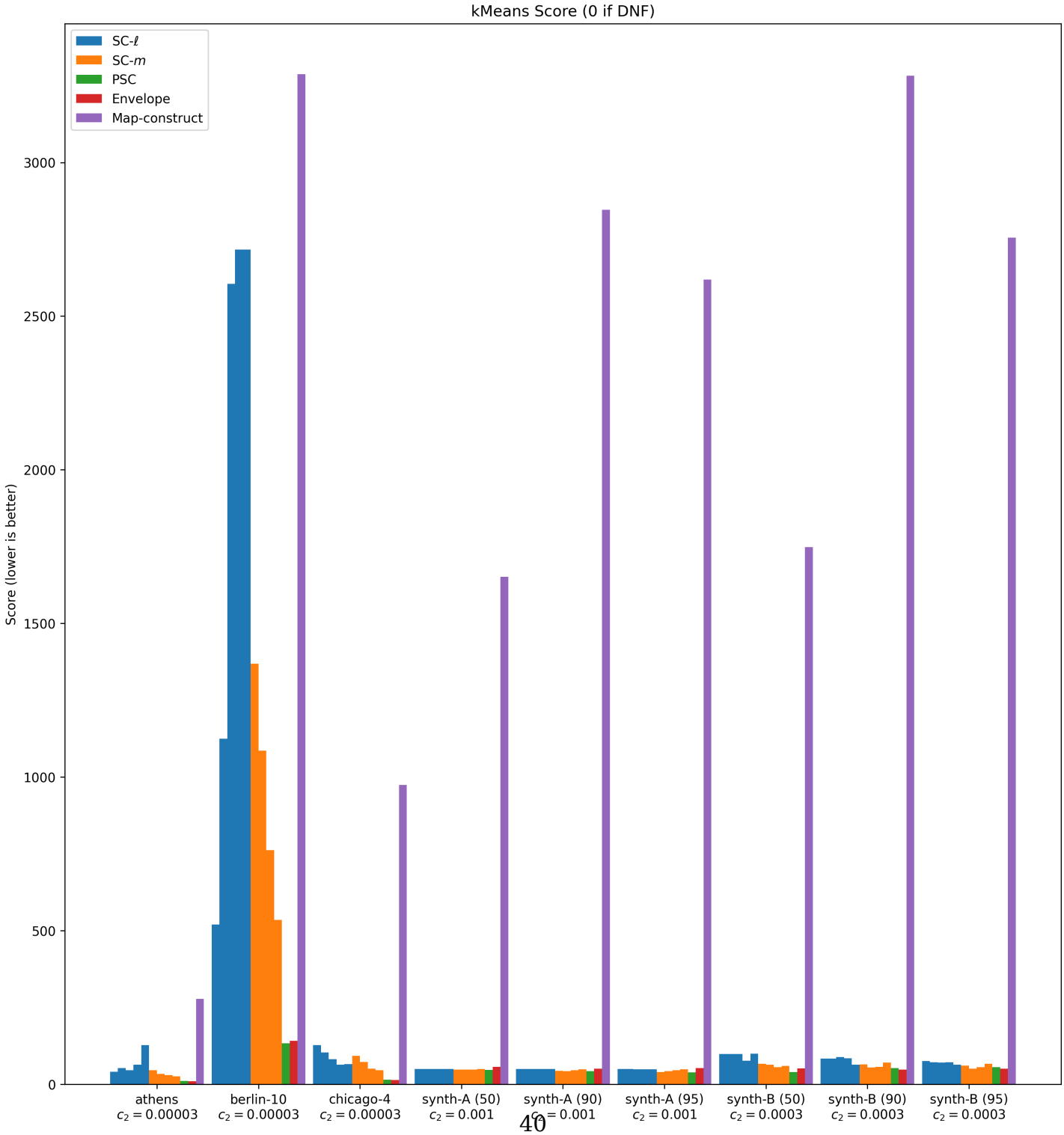


Figure 14: • k-means clustering • k-means score • Small c_2 •

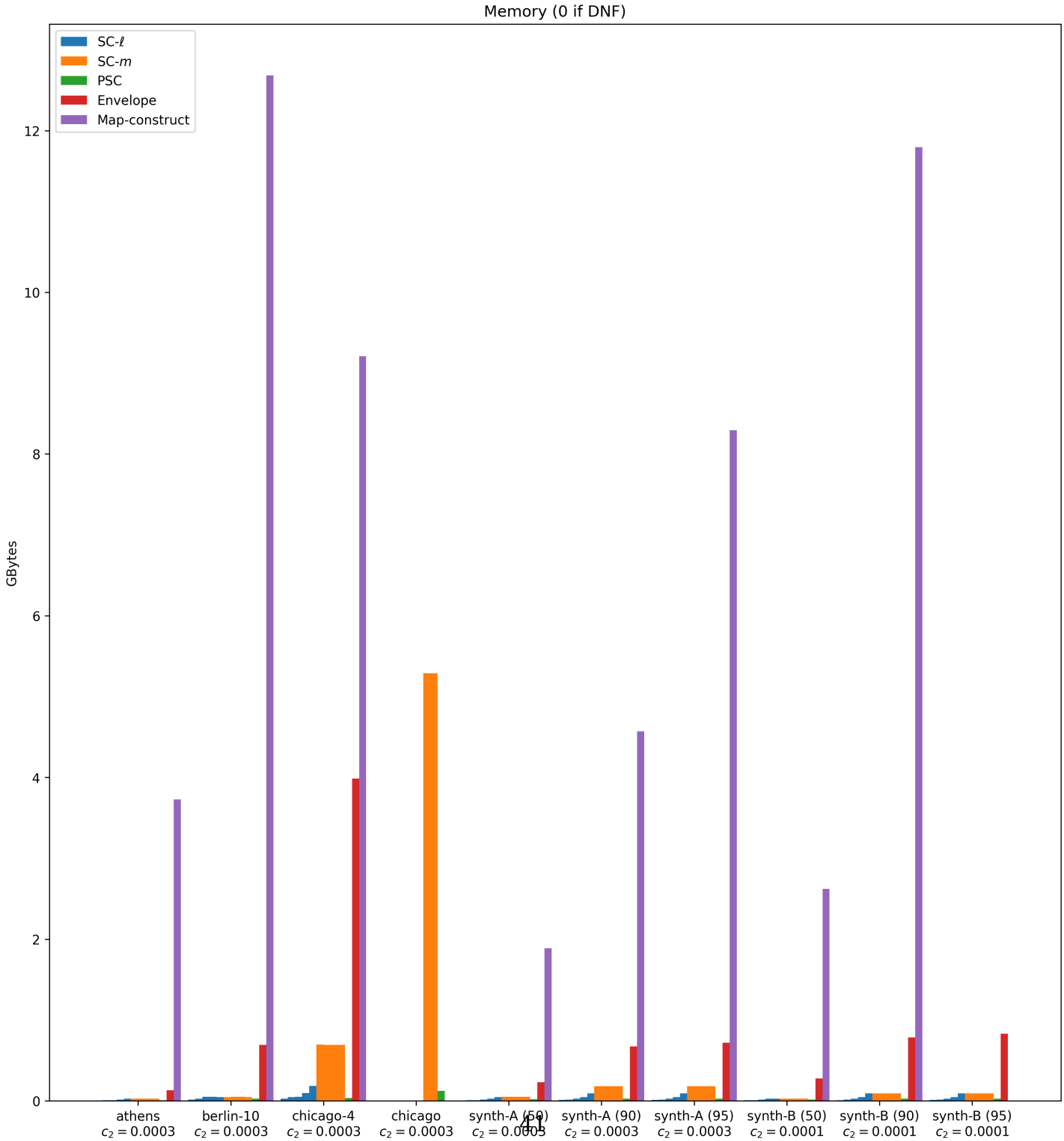


Figure 15: • k-means clustering • Memory usage • Medium c_2 •

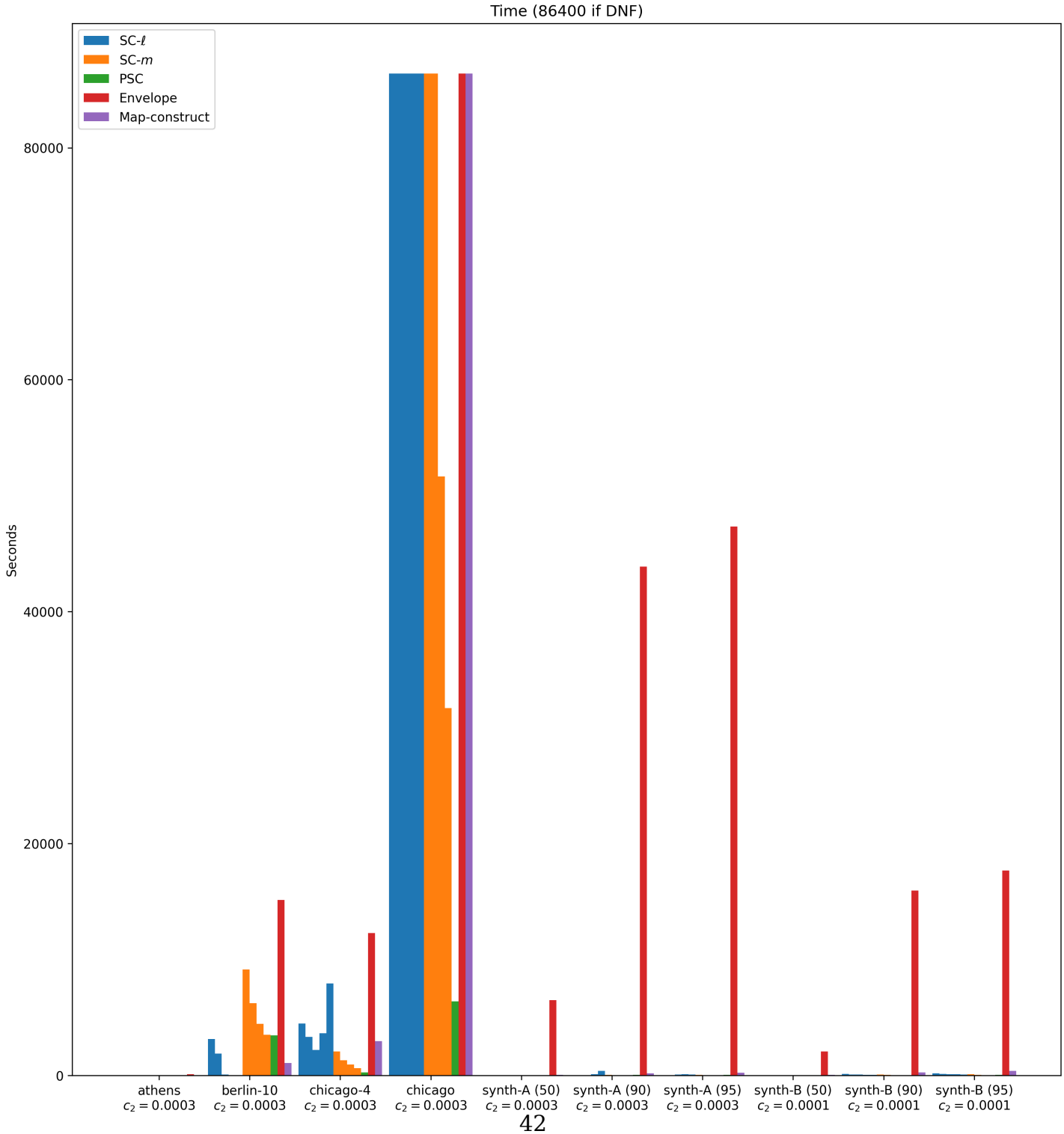


Figure 16: • k-means clustering • Running time • Medium c_2 •

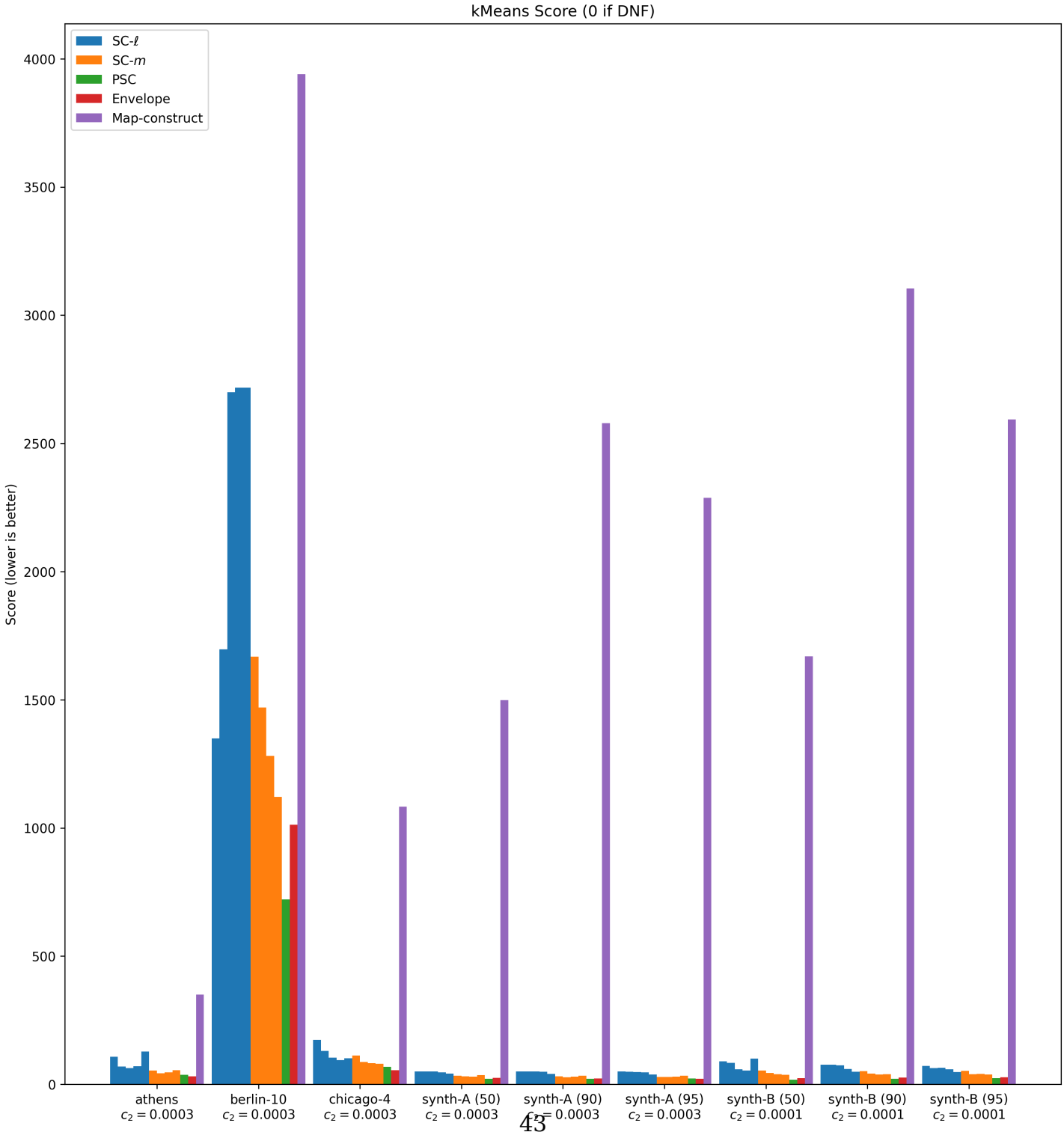


Figure 17: • k-means clustering • k-means score • Medium c_2 •

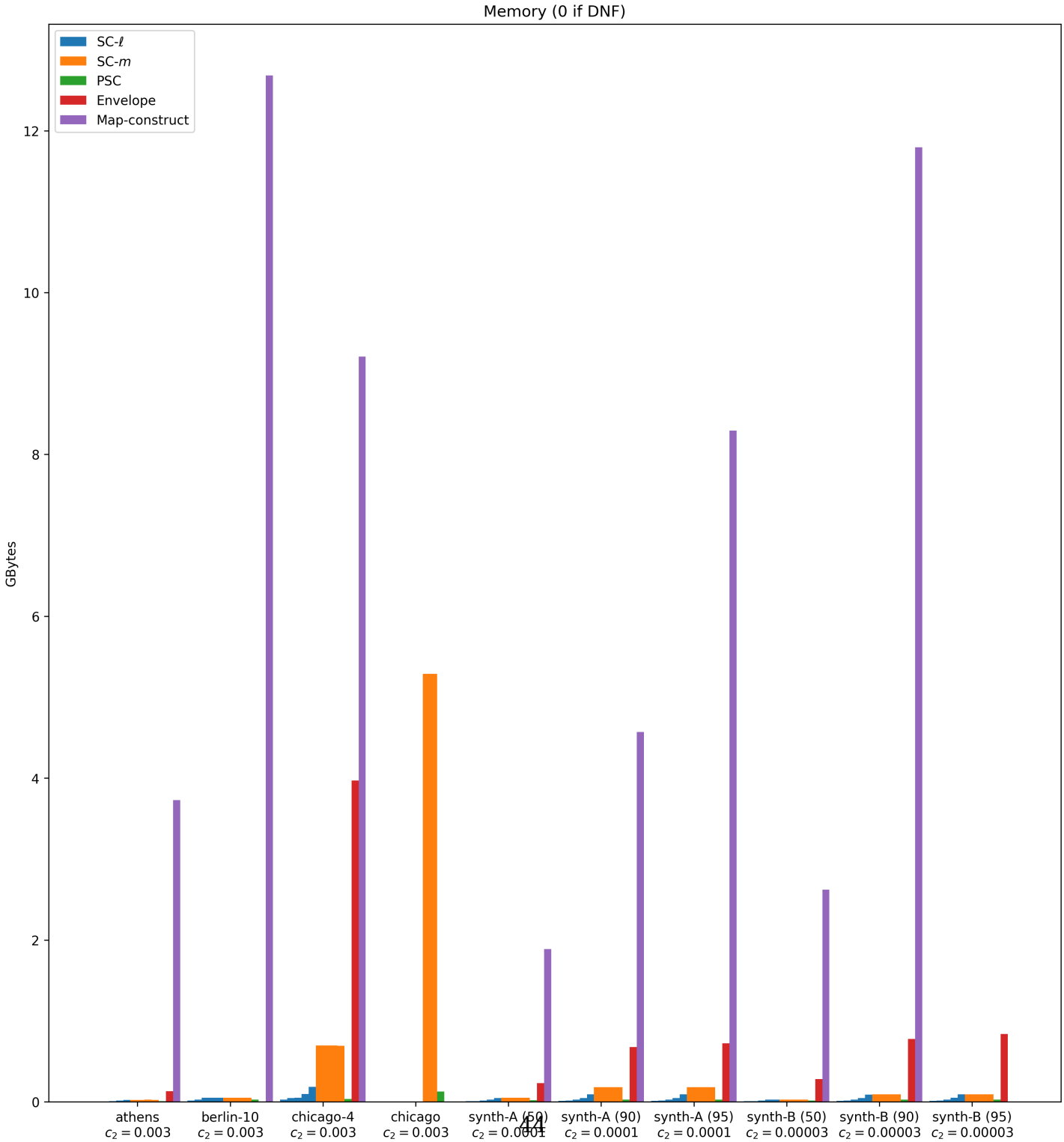


Figure 18: • k-means clustering • Memory usage • Large c_2 •

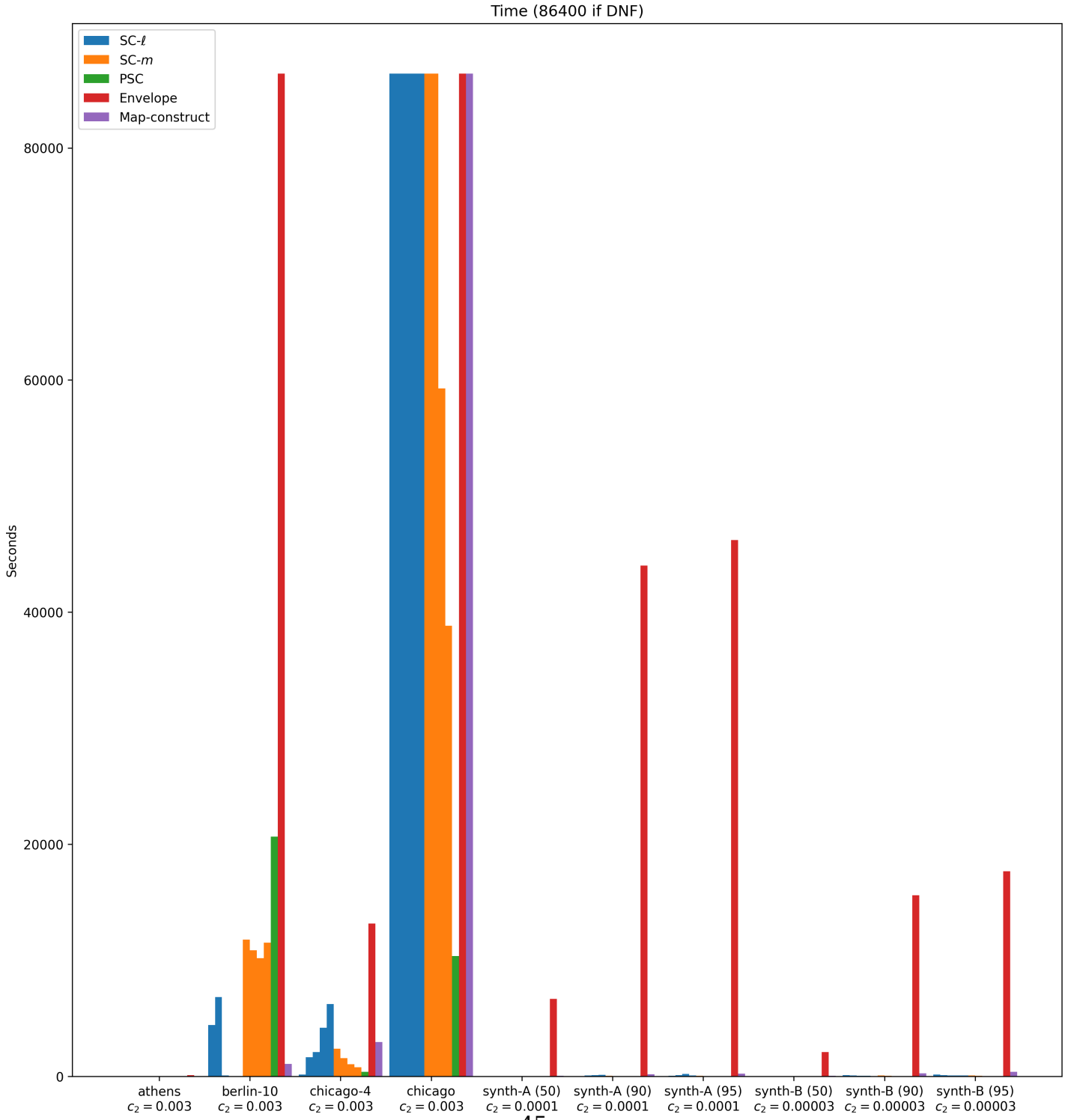


Figure 19: • k-means clustering • Running time • Large c_2 •

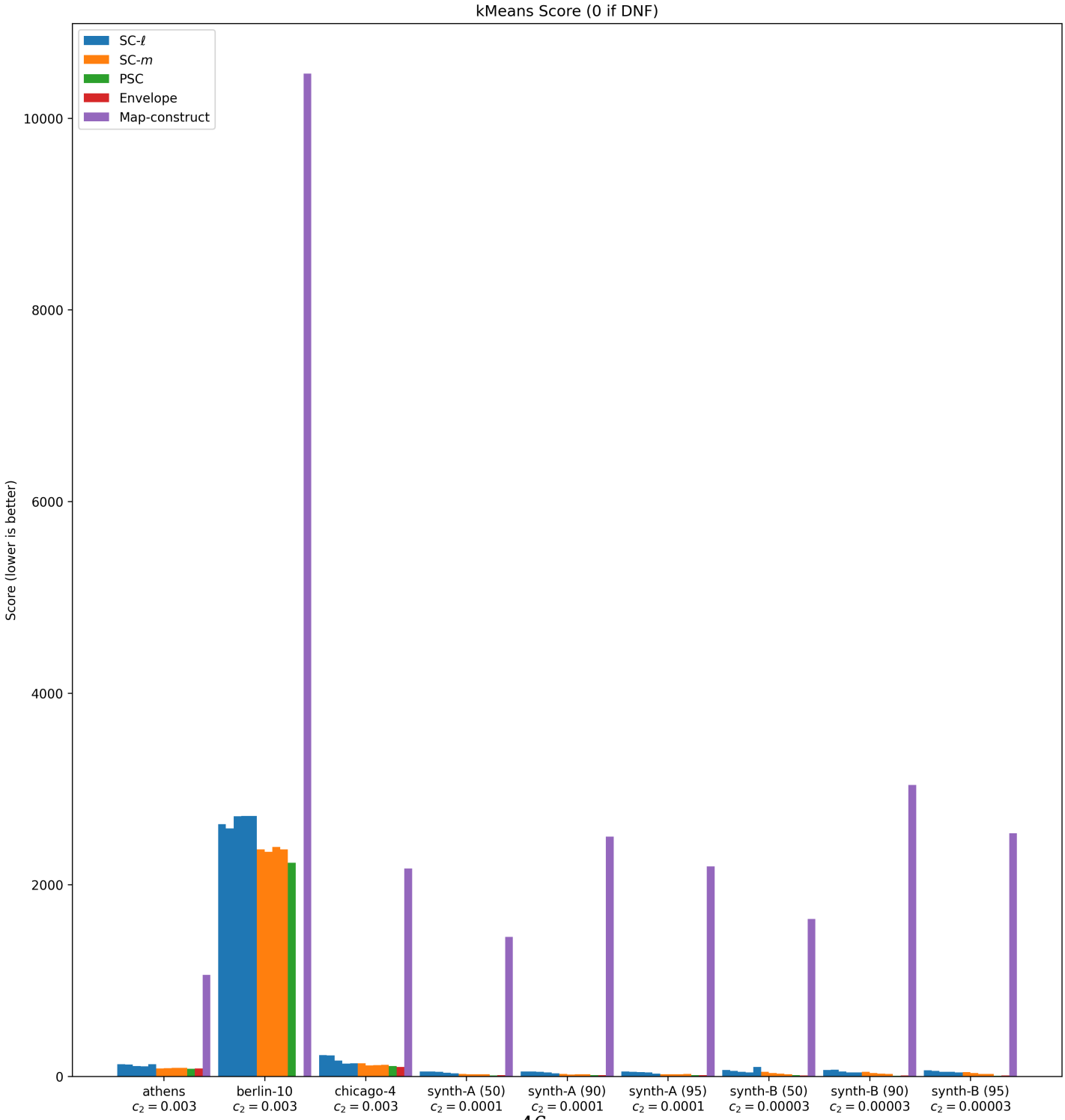


Figure 20: • k-means clustering • k-means score • Large c_2 •

C Raw data

In the following tables, we compare our implementations SC-$*$ and PSC, to the implementations presented in the literature. Specifically the Envelope implementation from [3] and the Map-construct implementation from [12]. Each table specifies whether we do k-means clustering or k-centre clustering. It then specifies the data set. We list time and space consumption, and, for each clustering, its score using the scoring vector. We also report additional data the reader may find interesting such as the number of clusters, or the maximum Fréchet distance between a subtrajectory in a cluster and its centre. We run our experiments on various data sets, including very large data. Consequently, some had timed out by our 24 hour deadline; for timed-out instances, we mark their row by $-$. In a few of the examples, the output is the empty clustering (and thus not all measures are well-defined); these can be recognised by the number of clusters being marked as 0. For these rows, other metrics may display exceptional values. E.g., then the maximum Fréchet distance is $-\infty$.

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	3.39	0.01	(1.0, 3e-05, 128)	15	1294.70	262.66	40.82	297	158.07
SC-ℓ-8	3.77	0.01	(1.0, 3e-05, 128)	16	1624.80	410.16	52.70	178	60.56
SC-ℓ-16	2.72	0.01	(1.0, 3e-05, 128)	11	1635.81	538.91	45.72	34	20.36
SC-ℓ-32	1.42	0.01	(1.0, 3e-05, 128)	4	818.49	514.21	63.86	40	23.00
SC-ℓ-64	0.48	0.03	(1.0, 3e-05, 128)	0	-inf	nan	128.00	-inf	nan
SC-m-2	4.44	0.03	(1.0, 3e-05, 128)	38	1584.88	399.76	46.26	3	2.89
SC-m-4	3.06	0.03	(1.0, 3e-05, 128)	26	3274.57	485.05	34.50	5	4.77
SC-m-8	2.04	0.03	(1.0, 3e-05, 128)	18	1638.00	736.98	30.05	9	8.06
SC-m-16	1.48	0.03	(1.0, 3e-05, 128)	14	2540.57	666.06	25.57	17	14.64
PSC	2.38	0.01	(1.0, 3e-05, 128)	6	2888.73	840.06	11.46	35	26.67
Envelope	125.38	0.13	(1.0, 3e-05, 128)	4	3890.66	1086.80	9.77	58	43.00
Map-construct	22.77	3.73	(1.0, 3e-05, 128)	261	501.25	134.79	278.41	41	8.17

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	9.18	0.01	(1.0, 0.0003, 128)	18	204.72	79.56	107.25	100	32.61
SC-ℓ-8	6.18	0.01	(1.0, 0.0003, 128)	23	204.39	121.54	69.47	33	12.35
SC-ℓ-16	3.16	0.01	(1.0, 0.0003, 128)	13	407.84	174.47	63.94	27	10.69
SC-ℓ-32	1.76	0.02	(1.0, 0.0003, 128)	7	815.73	362.93	70.85	19	9.86
SC-ℓ-64	0.48	0.03	(1.0, 0.0003, 128)	0	-inf	nan	128.00	-inf	nan
SC-m-2	4.85	0.03	(1.0, 0.0003, 128)	39	816.49	293.29	54.50	3	2.72
SC-m-4	3.17	0.03	(1.0, 0.0003, 128)	26	1323.31	298.50	42.90	5	4.27
SC-m-8	2.35	0.03	(1.0, 0.0003, 128)	18	1194.66	350.08	47.04	9	6.94
SC-m-16	2.23	0.03	(1.0, 0.0003, 128)	17	1634.40	399.85	54.76	17	7.12
PSC	4.93	0.01	(1.0, 0.0003, 128)	17	1779.26	306.56	36.66	24	7.18
Envelope	118.78	0.13	(1.0, 0.0003, 128)	12	1758.88	351.92	31.63	27	11.50
Map-construct	22.77	3.73	(1.0, 0.0003, 128)	261	501.25	134.79	349.52	41	8.17

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	1.86	0.00	(1.0, 0.003, 128)	1	23.21	18.60	127.61	14	14.00
SC-ℓ-8	6.66	0.01	(1.0, 0.003, 128)	8	99.68	52.72	124.75	11	6.75
SC-ℓ-16	5.45	0.01	(1.0, 0.003, 128)	16	200.97	103.75	106.81	11	5.12
SC-ℓ-32	3.58	0.01	(1.0, 0.003, 128)	10	334.83	136.42	104.49	7	2.80
SC-ℓ-64	0.48	0.03	(1.0, 0.003, 128)	0	-inf	nan	128.00	-inf	nan
SC-m-2	6.26	0.03	(1.0, 0.003, 128)	41	305.45	153.02	84.00	3	2.05
SC-m-4	5.40	0.03	(1.0, 0.003, 128)	32	261.43	152.28	85.83	5	2.47
SC-m-8	6.60	0.03	(1.0, 0.003, 128)	37	294.68	156.82	88.58	8	1.95
SC-m-16	6.28	0.03	(1.0, 0.003, 128)	35	305.45	182.85	87.93	8	1.91
PSC	12.73	0.01	(1.0, 0.003, 128)	28	224.47	147.84	80.07	8	2.96
Envelope	106.36	0.13	(1.0, 0.003, 128)	22	815.91	182.00	82.98	12	4.55
Map-construct	22.77	3.73	(1.0, 0.003, 128)	261	501.25	134.79	1060.63	41	8.17

Figure 21: ■ k-means clustering ■ Athens-small ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	3.54	0.01	(1.0, 0.003, 128)	1	3163.53	1403.34	10.49	2836	2836.00
SC-ℓ-8	2.76	0.01	(1.0, 0.003, 128)	1	3163.53	1778.20	10.49	2832	2832.00
SC-ℓ-16	1.84	0.01	(1.0, 0.003, 128)	1	3137.06	2039.87	10.41	2824	2824.00
SC-ℓ-32	1.08	0.02	(1.0, 0.003, 128	1	3259.12	2120.31	10.78	2806	2806.00
SC-ℓ-64	0.46	0.03	(1.0, 0.003, 128)	0	-inf	nan	128.00	-inf	nan
SC-m-2	4.72	0.01	(1.0, 0.003, 128)	4	1637.99	955.56	8.91	2497	703.75
SC-m-4	3.62	0.01	(1.0, 0.003, 128)	1	3163.53	1403.34	10.49	2836	2836.00
SC-m-8	2.65	0.01	(1.0, 0.003, 128)	1	3163.53	1778.20	10.49	2832	2832.00
SC-m-16	1.87	0.01	(1.0, 0.003, 128)	1	3137.06	2039.87	10.41	2824	2824.00
PSC	3.26	0.01	(1.0, 0.003, 128)	3	1638.28	1007.44	7.91	2511	944.00
Map-construct	24.10	4.61	(1.0, 0.03, 128)	261	501.25	134.79	285.55	41	8.17

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	3.64	0.01	(1.0, 0.03, 128)	5	818.50	440.71	33.34	1165	538.60
SC-ℓ-8	2.76	0.01	(1.0, 0.03, 128)	5	819.20	534.19	34.98	1050	486.20
SC-ℓ-16	1.86	0.01	(1.0, 0.03, 128)	2	1637.51	1032.72	54.96	1814	1192.50
SC-ℓ-32	1.09	0.02	(1.0, 0.03, 128)	5	409.54	377.89	85.30	38	14.40
SC-ℓ-64	0.46	0.03	(1.0, 0.03, 128)	0	-inf	nan	128.00	-inf	nan
SC-m-2	4.83	0.01	(1.0, 0.03, 128)	7	818.69	509.62	32.01	1212	392.57
SC-m-4	3.56	0.01	(1.0, 0.03, 128)	5	818.50	440.71	33.34	1165	538.60
SC-m-8	2.78	0.01	(1.0, 0.03, 128)	5	819.20	534.19	34.98	1050	486.20
SC-m-16	1.85	0.01	(1.0, 0.03, 128)	2	1637.51	1032.72	54.96	1814	1192.50
PSC	3.10	0.01	(1.0, 0.03, 128	15	409.48	245.04	32.29	220	78.47
Map-construct	24.10	4.61	(1.0, 0.3, 128)	261	501.25	134.79	420.88	41	8.17

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	3.71	0.01	(1.0, 0.3, 128)	22	204.77	142.53	114.08	270	68.82
SC-ℓ-8	2.65	0.01	(1.0, 0.3, 128)	16	204.79	166.51	119.35	201	31.00
SC-ℓ-16	1.94	0.01	(1.0, 0.3, 128)	13	204.47	163.21	122.48	33	10.15
SC-ℓ-32	1.09	0.02	(1.0, 0.3, 128)	25	0.00	nan	115.82	1	1.00
SC-ℓ-64	0.49	0.03	(1.0, 0.3, 128)	0	-inf	nan	128.00	-inf	nan
SC-m-2	4.83	0.01	(1.0, 0.3, 128)	24	204.77	123.97	107.79	316	88.17
SC-m-4	3.57	0.01	(1.0, 0.3, 128)	22	204.77	142.53	114.08	270	68.82
SC-m-8	2.77	0.01	(1.0, 0.3, 128)	16	204.79	166.51	119.35	201	31.00
SC-m-16	1.85	0.01	(1.0, 0.3, 128)	13	204.47	163.21	122.48	33	10.15
PSC	3.06	0.01	(1.0, 0.3, 128)	35	102.36	62.52	95.32	78	28.74
Map-construct	24.10	4.61	(1.0, 3.0, 128)	261	501.25	134.79	1774.26	41	8.17

Figure 22: ■ k-centre clustering ■ Athens-small ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	84686.70	0.20	(1.0, 3e-05, 888)	71	339.70	89.15	206.50	970	117.28
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	80331.20	5.29	(1.0, 3e-05, 888)	198	1343.00	228.94	227.66	5	4.59
SC-m-8	49483.80	5.29	(1.0, 3e-05, 888)	124	1356.23	182.11	172.73	9	8.48
SC-m-16	30217.30	5.29	(1.0, 3e-05, 888)	90	1356.41	184.20	138.25	17	15.59
PSC	3553.34	0.12	(1.0, 3e-05, 888)	23	1355.38	294.04	44.47	191	91.70
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	51656.80	5.29	(1.0, 0.0003, 888)	139	1018.31	73.99	228.73	9	8.23
SC-m-16	31688.40	5.29	(1.0, 0.0003, 888)	108	664.44	43.02	215.16	17	16.16
PSC	6381.29	0.13	(1.0, 0.0003, 888)	86	1478.85	38.53	152.27	175	36.24
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	59279.00	5.29	(1.0, 0.003, 888)	138	84.69	22.71	338.81	9	8.18
SC-m-16	38831.10	5.29	(1.0, 0.003, 888)	98	84.23	20.57	342.55	17	15.66
PSC	10375.70	0.13	(1.0, 0.003, 888)	96	84.90	20.40	308.53	66	17.05
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Figure 23: ■ k-means clustering ■ Chicago ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	22670.20	0.21	(1.0, 10.0, 888)	147	21.24	14.78	529.41	359	122.15
SC-ℓ-8	25517.70	0.28	(1.0, 10.0, 888)	96	21.24	15.36	504.91	327	105.07
SC-ℓ-16	26519.10	0.30	(1.0, 10.0, 888)	73	21.24	16.20	526.67	260	70.49
SC-ℓ-32	47077.50	0.48	(1.0, 10.0, 888)	59	21.24	16.68	556.16	218	42.02
SC-ℓ-64	75286.00	0.81	(1.0, 10.0, 888)	42	21.22	17.27	675.72	151	23.43
SC-m-2	20086.90	0.17	(1.0, 10.0, 888)	218	21.24	13.90	586.90	373	140.00
SC-m-4	23542.20	0.21	(1.0, 10.0, 888)	147	21.24	14.78	529.41	359	122.15
SC-m-8	26235.70	0.28	(1.0, 10.0, 888)	96	21.24	15.36	504.91	327	105.07
SC-m-16	24774.00	0.30	(1.0, 10.0, 888)	73	21.24	16.20	526.67	260	70.49
PSC	2987.91	0.10	(1.0, 10.0, 888)	95	21.23	15.90	397.87	218	50.75

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	12192.90	0.21	(1.0, 1.0, 888)	95	84.97	63.44	231.57	3018	758.14
SC-ℓ-8	12092.50	0.28	(1.0, 1.0, 888)	86	84.97	71.45	243.16	1731	219.01
SC-ℓ-16	12774.90	0.30	(1.0, 1.0, 888)	59	84.97	72.65	244.42	367	99.17
SC-ℓ-32	24889.30	0.48	(1.0, 1.0, 888)	43	84.97	72.62	274.79	236	68.33
SC-ℓ-64	46779.30	0.81	(1.0, 1.0, 888)	36	42.47	34.08	444.17	156	30.00
SC-m-2	12277.50	0.17	(1.0, 1.0, 888)	97	84.97	53.62	219.15	3725	981.74
SC-m-4	13744.90	0.21	(1.0, 1.0, 888)	95	84.97	63.44	231.57	3018	758.14
SC-m-8	13305.30	0.28	(1.0, 1.0, 888)	86	84.97	71.45	243.16	1731	219.01
SC-m-16	12344.80	0.30	(1.0, 1.0, 888)	59	84.97	72.65	244.42	367	99.17
PSC	2767.08	0.10	(1.0, 1.0, 888)	75	42.49	31.83	167.75	191	47.93

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	9884.09	0.21	(1.0, 0.1, 888)	24	339.89	215.40	60.17	21441	4553.62
SC-ℓ-8	10658.20	0.28	(1.0, 0.1, 888)	22	339.90	224.77	60.56	19986	4653.41
SC-ℓ-16	9925.72	0.30	(1.0, 0.1, 888)	23	339.89	259.59	77.27	17183	3878.83
SC-ℓ-32	21052.60	0.48	(1.0, 0.1, 888)	25	339.89	295.97	109.17	12010	1625.88
SC-ℓ-64	46322.50	0.81	(1.0, 0.1, 888)	3	1359.58	911.08	140.40	75493	33602.33
SC-m-2	10749.60	0.17	(1.0, 0.1, 888)	21	339.90	207.19	57.98	22026	5411.86
SC-m-4	11364.30	0.21	(1.0, 0.1, 888)	24	339.89	215.40	60.17	21441	4553.62
SC-m-8	10211.10	0.28	(1.0, 0.1, 888)	22	339.90	224.77	60.56	19986	4653.41
SC-m-16	10222.00	0.30	(1.0, 0.1, 888)	23	339.89	259.59	77.27	17183	3878.83
PSC	3035.34	0.10	(1.0, 0.1, 888)	25	339.89	242.66	61.35	6255	1404.08

Figure 24: ■ k-centre clustering ■ Chicago ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	2510.48	0.03	(1.0, 3e-05, 222.0)	57	339.82	83.16	127.77	752	261.81
SC-ℓ-8	3011.34	0.05	(1.0, 3e-05, 222.0)	63	1359.43	80.09	103.54	409	67.78
SC-ℓ-16	1767.85	0.05	(1.0, 3e-05, 222.0)	39	339.72	66.33	82.10	135	36.54
SC-ℓ-32	3226.07	0.10	(1.0, 3e-05, 222.0)	24	1358.95	202.53	64.15	95	34.92
SC-ℓ-64	6082.50	0.19	(1.0, 3e-05, 222.0)	17	1358.64	329.34	66.33	75	29.47
SC-m-2	1790.72	0.69	(1.0, 3e-05, 222.0)	79	1334.43	354.28	92.39	3	2.73
SC-m-4	1199.72	0.69	(1.0, 3e-05, 222.0)	51	1355.97	297.86	73.05	5	4.61
SC-m-8	745.31	0.69	(1.0, 3e-05, 222.0)	35	1353.05	403.78	51.14	9	7.71
SC-m-16	534.49	0.69	(1.0, 3e-05, 222.0)	26	1359.13	503.27	45.68	17	11.96
PSC	88.13	0.03	(1.0, 3e-05, 222.0)	7	2641.63	673.79	15.03	67	43.14
Envelope	11871.47	4.03	(1.0, 3e-05, 222.0)	6	2552.42	777.98	14.17	105	58.50
Map-construct	2972.20	9.21	(1.0, 3e-05, 222.0)	957	156.53	27.25	971.96	74	15.82

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	4487.10	0.03	(1.0, 0.0003, 222)	68	42.25	15.20	173.01	90	46.96
SC-ℓ-8	3339.71	0.05	(1.0, 0.0003, 222)	63	42.46	18.82	130.53	83	36.73
SC-ℓ-16	2209.07	0.05	(1.0, 0.0003, 222)	43	84.73	25.63	103.96	79	30.28
SC-ℓ-32	3632.63	0.10	(1.0, 0.0003, 222)	32	338.50	36.39	94.84	49	21.06
SC-ℓ-64	7949.50	0.19	(1.0, 0.0003, 222)	28	339.86	118.87	102.13	38	13.43
SC-m-2	2075.07	0.69	(1.0, 0.0003, 222)	91	679.48	213.15	112.83	3	2.40
SC-m-4	1312.14	0.69	(1.0, 0.0003, 222)	54	1359.07	131.40	87.47	5	4.39
SC-m-8	948.43	0.69	(1.0, 0.0003, 222)	44	2421.63	108.58	82.98	9	7.86
SC-m-16	640.77	0.69	(1.0, 0.0003, 222)	25	679.65	94.89	80.06	17	15.52
PSC	271.15	0.04	(1.0, 0.0003, 222)	38	1863.08	78.54	67.76	44	16.13
Envelope	12282.06	3.98	(1.0, 0.0003, 222)	19	2533.25	232.98	55.39	38	24.53
Map-construct	2972.20	9.21	(1.0, 0.0003, 222)	957	156.53	27.25	1080.57	74	15.82

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	173.23	0.03	(1.0, 0.003, 222)	0	-inf	nan	222.00	-inf	nan
SC-ℓ-8	1664.45	0.05	(1.0, 0.003, 222)	7	21.23	14.60	217.77	74	54.29
SC-ℓ-16	2107.72	0.05	(1.0, 0.003, 222)	36	21.24	16.16	165.13	63	31.50
SC-ℓ-32	4187.72	0.10	(1.0, 0.003, 222)	28	42.32	17.69	133.72	49	20.96
SC-ℓ-64	6232.40	0.19	(1.0, 0.003, 222)	22	42.40	20.59	135.58	36	11.36
SC-m-2	2396.72	0.69	(1.0, 0.003, 222)	96	108.68	30.94	137.59	3	2.12
SC-m-4	1569.89	0.69	(1.0, 0.003, 222)	57	84.82	24.67	116.17	5	3.96
SC-m-8	1062.67	0.69	(1.0, 0.003, 222)	38	42.45	22.12	117.21	9	7.45
SC-m-16	790.13	0.69	(1.0, 0.003, 222)	26	41.52	19.39	121.90	17	12.73
PSC	405.62	0.04	(1.0, 0.003, 222)	38	83.71	21.15	108.81	38	9.05
Envelope	13172.97	3.97	(1.0, 0.003, 222)	34	316.20	31.26	99.36	24	10.68
Map-construct	2972.20	9.21	(1.0, 0.003, 222)	957	156.53	27.25	2166.58	74	15.82

Figure 25: ■ k-means clustering ■ Chicago-4 ■

Name	Seconds	GBytes	$c2$	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	431.41	0.05	(1.0, 0.1, 222)	17	339.87	215.52	53.01	5453	1577.12
SC-ℓ-8	385.76	0.07	(1.0, 0.1, 222)	18	339.88	221.10	55.57	5086	1404.22
SC-ℓ-16	309.62	0.08	(1.0, 0.1, 222)	19	339.90	259.34	60.40	4352	1146.68
SC-ℓ-32	502.92	0.12	(1.0, 0.1, 222)	20	339.89	300.27	66.19	2006	396.90
SC-ℓ-64	1382.97	0.21	(1.0, 0.1, 222)	5	679.79	515.84	90.98	7280	2985.60
SC-m-2	528.91	0.05	(1.0, 0.1, 222)	16	339.89	208.02	52.46	5602	1745.44
SC-m-4	426.38	0.05	(1.0, 0.1, 222)	17	339.87	215.52	53.01	5453	1577.12
SC-m-8	383.91	0.07	(1.0, 0.1, 222)	18	339.88	221.10	55.57	5086	1404.22
SC-m-16	327.37	0.08	(1.0, 0.1, 222)	19	339.90	259.34	60.40	4352	1146.68
PSC	141.25	0.03	(1.0, 0.1, 222)	19	339.90	238.42	56.04	1542	496.32

Name	Seconds	GBytes	$c2$	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	428.39	None	(1.0, 1.0, 222)	67	84.97	64.04	177.58	736	236.34
SC-ℓ-8	350.93	0.07	(1.0, 1.0, 222)	64	42.48	32.84	143.87	90	36.81
SC-ℓ-16	307.23	0.08	(1.0, 1.0, 222)	41	21.24	16.19	131.86	64	28.85
SC-ℓ-32	612.27	0.12	(1.0, 1.0, 222)	27	21.24	16.65	129.96	49	20.93
SC-ℓ-64	1791.53	0.21	(1.0, 1.0, 222)	20	21.24	17.48	152.83	36	11.30
SC-m-2	532.90	0.04	(1.0, 1.0, 222)	67	84.97	54.20	177.56	928	313.94
SC-m-4	431.68	0.05	(1.0, 1.0, 222)	67	84.97	64.04	177.58	736	236.34
SC-m-8	361.35	0.07	(1.0, 1.0, 222)	64	42.48	32.84	143.87	90	36.81
SC-m-16	399.70	0.08	(1.0, 1.0, 222)	41	21.24	16.19	131.86	64	28.85
PSC	136.59	0.03	(1.0, 1.0, 222)	41	21.24	16.67	108.44	44	19.61

Name	Seconds	GBytes	$c2$	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	599.88	0.05	(1.0, 10.0, 222)	0	-inf	nan	222.00	-inf	nan
SC-ℓ-8	726.98	0.07	(1.0, 10.0, 222)	0	-inf	nan	222.00	-inf	nan
SC-ℓ-16	448.36	0.08	(1.0, 10.0, 222)	0	-inf	nan	222.00	-inf	nan
SC-ℓ-32	1269.39	0.12	(1.0, 10.0, 222)	0	-inf	nan	222.00	-inf	nan
SC-ℓ-64	2397.99	0.21	(1.0, 10.0, 222)	0	-inf	nan	222.00	-inf	nan
SC-m-2	560.50	0.04	(1.0, 10.0, 222)	0	-inf	nan	222.00	-inf	nan
SC-m-4	489.27	0.05	(1.0, 10.0, 222)	0	-inf	nan	222.00	-inf	nan
SC-m-8	590.26	0.07	(1.0, 10.0, 222)	0	-inf	nan	222.00	-inf	nan
SC-m-16	562.17	0.08	(1.0, 10.0, 222)	0	-inf	nan	222.00	-inf	nan
PSC	146.09	0.03	(1.0, 10.0, 222.0)	76	0.00	nan	199.97	2	1.01

Figure 26: ■ k-centre clustering ■ Chicago-4 ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	17949.40	2.16	(1.0, 0.01, 2011)	165	57.15	9.10	432.75	36	14.02
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Figure 27: ■ k-means clustering ■ Drifter ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	78998.10	1.62	(1.0, 10.0, 2011)	73	16.38	10.77	166090432.35	157991	21132.48
PSC	78657.30	1.62	(1.0, 30.0, 2011)	174	8.19	5.66	197774272.29	74926	6698.74
PSC	85820.10	1.49	(1.0, 100.0, 2011)	398	4.10	2.91	241716795.73	28596	2089.96
PSC	81131.30	1.62	(1.0, 300.0, 2011)	1073	0.00	nan	1460.38	1	1.00
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Figure 28: ■ k-centre clustering ■ Drifter ■

Name	Seconds	GBytes	$c2$	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	287.47	0.30	(1.0, 0.001, 362)	10	104.73	19.12	19.15	79	46.40
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Name	Seconds	GBytes	$c2$	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	764.06	0.30	(1.0, 0.01, 362)	34	34.95	6.86	73.01	28	13.26
Envelope	—	—	—	—	—	—	—	—	—
Map Construct	—	—	—	—	—	—	—	—	—

Name	Seconds	GBytes	$c2$	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	2183.53	0.30	(1.0, 0.1, 362)	80	6.55	1.67	174.96	20	3.70
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Figure 29: ■ k-means clustering ■ UniD ■

Name	Seconds	GBytes	$c2$	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	55043.80	0.25	(1.0, 3.0, 362)	8	13.11	8.10	49.04	52548	26513.50
SC-ℓ-8	63151.50	0.24	(1.0, 3.0, 362)	8	13.11	8.15	48.95	52240	26399.12
SC-ℓ-16	68155.60	0.42	(1.0, 3.0, 362)	9	13.11	8.21	49.69	51592	23293.78
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-4	67209.00	0.26	(1.0, 3.0, 362)	27	6.55	4.10	49.06	21096	7827.89
SC-m-8	55189.10	0.24	(1.0, 3.0, 362)	8	13.11	8.10	49.04	52548	26513.50
SC-m-16	59843.10	0.24	(1.0, 3.0, 362)	8	13.11	8.15	48.95	52240	26399.12
SC-m-32	72703.90	0.42	(1.0, 3.0, 362)	9	13.11	8.21	49.69	51592	23293.78
PSC	1833.13	0.21	(1.0, 3.0, 362)	10	13.11	8.43	50.69	47466	18912.10
Map-construct	—	—	—	—	—	—	—	—	—

Name	Seconds	GBytes	$c2$	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	44324.10	0.47	(1.0, 30.0, 362)	68	3.28	2.11	185.63	9322	2911.51
SC-ℓ-8	37831.30	0.49	(1.0, 30.0, 362)	71	3.28	2.13	186.04	9261	2758.72
SC-ℓ-16	45932.90	0.66	(1.0, 30.0, 362)	70	3.28	2.14	187.94	9129	2704.01
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	46785.30	0.40	(1.0, 30.0, 362)	72	3.28	2.11	186.06	9353	2807.14
SC-m-4	44663.20	0.47	(1.0, 30.0, 362)	68	3.28	2.11	185.63	9322	2911.51
SC-m-8	39783.20	0.49	(1.0, 30.0, 362)	71	3.28	2.13	186.04	9261	2758.72
SC-m-16	46280.00	0.66	(1.0, 30.0, 362)	70	3.28	2.14	187.94	9129	2704.01
PSC	1811.57	0.21	(1.0, 30.0, 362)	61	3.28	2.22	186.02	8092	794.84
Map-construct	—	—	—	—	—	—	—	—	—

Name	Seconds	GBytes	$c2$	Clusters	Max Frechet	Avg Frechet	kCenters	Max Size	Avg Size
SC-ℓ-4	53806.00	0.31	(1.0, 300.0, 362)	10	0.00	0.00	353.41	2071	1148.10
SC-ℓ-8	59203.00	0.25	(1.0, 300.0, 362)	10	0.00	0.00	353.41	2032	1134.10
SC-ℓ-16	65860.00	0.41	(1.0, 300.0, 362)	8	0.00	0.00	353.55	1996	1207.50
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	56941.30	0.28	(1.0, 300.0, 362)	10	0.00	0.00	353.41	2082	1153.40
SC-m-4	53550.40	0.30	(1.0, 300.0, 362)	10	0.00	0.00	353.41	2071	1148.10
SC-m-8	52683.50	0.24	(1.0, 300.0, 362)	10	0.00	0.00	353.41	2032	1134.10
SC-m-16	67746.20	0.43	(1.0, 300.0, 362)	8	0.00	0.00	353.55	1996	1207.50
PSC	1686.52	0.21	(1.0, 300.0, 362)	135	0.00	nan	273.17	1	1.00
Map-construct	—	—	—	—	—	—	—	—	—

Figure 30: ■ k-centre clustering ■ UniD ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	3.08	0.01	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	3.06	0.01	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-16	3.22	0.02	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-32	4.14	0.03	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-64	9.75	0.05	(1.0, 0.001, 50)	1	518.10	425.55	49.80	4	4.00
SC-m-2	18.13	0.05	(1.0, 0.001, 50)	35	951.62	720.54	47.58	2	1.43
SC-m-4	16.68	0.05	(1.0, 0.001, 50)	34	936.01	673.83	48.27	4	1.47
SC-m-8	17.53	0.05	(1.0, 0.001, 50)	36	892.24	716.90	47.60	4	1.39
SC-m-16	29.69	0.05	(1.0, 0.001, 50)	47	882.42	851.20	49.79	2	1.04
PSC	52.54	0.02	(1.0, 0.001, 50)	14	913.07	721.77	46.79	4	2.29
Envelope	6640.05	0.23	(1.0, 0.001, 50)	6	1778.38	1074.17	56.45	16	8.33
Map-construct	55.43	1.89	(1.0, 0.001, 50)	1431	817.76	36.36	1654.28	18	5.13

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	3.18	0.01	(1.0, 0.0003, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	3.17	0.01	(1.0, 0.0003, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-16	3.09	0.02	(1.0, 0.0003, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-32	14.24	0.03	(1.0, 0.0003, 50)	5	535.35	494.06	47.23	17	13.60
SC-ℓ-64	21.25	0.05	(1.0, 0.0003, 50)	12	1070.33	836.97	42.11	9	5.33
SC-m-2	12.21	0.05	(1.0, 0.0003, 50)	21	1063.61	932.75	33.11	3	2.38
SC-m-4	8.56	0.05	(1.0, 0.0003, 50)	15	1068.51	937.33	30.75	5	3.33
SC-m-8	6.75	0.05	(1.0, 0.0003, 50)	10	1063.83	875.74	29.73	8	5.00
SC-m-16	6.81	0.05	(1.0, 0.0003, 50)	12	1071.65	998.84	36.50	10	4.25
PSC	14.48	0.02	(1.0, 0.0003, 50)	7	2143.57	985.30	22.11	14	7.29
Envelope	6486.82	0.23	(1.0, 0.0003, 50)	3	1924.87	1297.20	25.08	40	17.67
Map-construct	55.43	1.89	(1.0, 0.0003, 50)	1431	817.76	36.36	1501.10	18	5.13

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	3.24	0.01	(1.0, 0.0001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	3.29	0.01	(1.0, 0.0001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-16	13.92	0.02	(1.0, 0.0001, 50)	5	536.11	404.39	47.66	36	23.20
SC-ℓ-32	13.89	0.03	(1.0, 0.0001, 50)	8	1061.05	576.50	38.16	23	16.00
SC-ℓ-64	16.02	0.05	(1.0, 0.0001, 50)	8	1069.96	958.22	30.63	19	10.25
SC-m-2	11.24	0.05	(1.0, 0.0001, 50)	20	2077.13	1009.06	26.63	3	2.65
SC-m-4	7.85	0.05	(1.0, 0.0001, 50)	13	1816.18	990.90	22.03	5	4.31
SC-m-8	5.86	0.05	(1.0, 0.0001, 50)	8	1065.53	922.80	21.57	9	7.88
SC-m-16	4.23	0.05	(1.0, 0.0001, 50)	5	1862.33	1001.98	23.97	16	11.20
PSC	12.85	0.02	(1.0, 0.0001, 50)	4	1809.92	1149.63	10.90	17	12.75
Envelope	6689.78	0.23	(1.0, 0.0001, 50)	3	1949.27	1359.89	11.66	36	20.67
Map-construct	55.43	1.89	(1.0, 0.0001, 50)	1431	817.76	36.36	1457.34	18	5.13

Figure 31: ■ k-means clustering ■ Berlin-10 ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	—	—	—	—	—	—	—	—	—
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	—	—	—	—	—	—	—	—	—
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	—	—	—	—	—	—	—	—	—
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Figure 32: ■ k-means clustering ■ Berlin ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	—	—	—	—	—	—	—	—	—
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	—	—	—	—	—	—	—	—	—
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	—	—	—	—	—	—	—	—	—
SC-ℓ-8	—	—	—	—	—	—	—	—	—
SC-ℓ-16	—	—	—	—	—	—	—	—	—
SC-ℓ-32	—	—	—	—	—	—	—	—	—
SC-ℓ-64	—	—	—	—	—	—	—	—	—
SC-m-2	—	—	—	—	—	—	—	—	—
SC-m-4	—	—	—	—	—	—	—	—	—
SC-m-8	—	—	—	—	—	—	—	—	—
SC-m-16	—	—	—	—	—	—	—	—	—
PSC	—	—	—	—	—	—	—	—	—
Envelope	—	—	—	—	—	—	—	—	—
Map-construct	—	—	—	—	—	—	—	—	—

Figure 33: ■ k-centre clustering ■ Berlin ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	3.08	0.01	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	3.06	0.01	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-16	3.22	0.02	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-32	4.14	0.03	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-64	9.75	0.05	(1.0, 0.001, 50)	1	518.10	425.55	49.80	4	4.00
SC-m-2	18.13	0.05	(1.0, 0.001, 50)	35	951.62	720.54	47.58	2	1.43
SC-m-4	16.68	0.05	(1.0, 0.001, 50)	34	936.01	673.83	48.27	4	1.47
SC-m-8	17.53	0.05	(1.0, 0.001, 50)	36	892.24	716.90	47.60	4	1.39
SC-m-16	29.69	0.05	(1.0, 0.001, 50)	47	882.42	851.20	49.79	2	1.04
PSC	52.54	0.02	(1.0, 0.001, 50)	14	913.07	721.77	46.79	4	2.29
Envelope	6640.05	0.23	(1.0, 0.001, 50)	6	1778.38	1074.17	56.45	16	8.33
Map-construct	55.43	1.89	(1.0, 0.001, 50)	1431	817.76	36.36	1651.56	18	5.13

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	3.18	0.01	(1.0, 0.0003, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	3.17	0.01	(1.0, 0.0003, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-16	3.09	0.02	(1.0, 0.0003, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-32	14.24	0.03	(1.0, 0.0003, 50)	5	535.35	494.06	47.23	17	13.60
SC-ℓ-64	21.25	0.05	(1.0, 0.0003, 50)	12	1070.33	836.97	42.11	9	5.33
SC-m-2	12.21	0.05	(1.0, 0.0003, 50)	21	1063.61	932.75	33.11	3	2.38
SC-m-4	8.56	0.05	(1.0, 0.0003, 50)	15	1068.51	937.33	30.75	5	3.33
SC-m-8	6.75	0.05	(1.0, 0.0003, 50)	10	1063.83	875.74	29.73	8	5.00
SC-m-16	6.81	0.05	(1.0, 0.0003, 50)	12	1071.65	998.84	36.50	10	4.25
PSC	14.48	0.02	(1.0, 0.0003, 50)	7	2143.57	985.30	22.11	14	7.29
Envelope	6486.82	0.23	(1.0, 0.0003, 50)	3	1924.87	1297.20	25.08	40	17.67
Map-construct	55.43	1.89	(1.0, 0.0003, 50)	1431	817.76	36.36	1498.39	18	5.13

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	3.24	0.01	(1.0, 0.0001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	3.29	0.01	(1.0, 0.0001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-16	13.92	0.02	(1.0, 0.0001, 50)	5	536.11	404.39	47.66	36	23.20
SC-ℓ-32	13.89	0.03	(1.0, 0.0001, 50)	8	1061.05	576.50	38.16	23	16.00
SC-ℓ-64	16.02	0.05	(1.0, 0.0001, 50)	8	1069.96	958.22	30.63	19	10.25
SC-m-2	11.24	0.05	(1.0, 0.0001, 50)	20	2077.13	1009.06	26.63	3	2.65
SC-m-4	7.85	0.05	(1.0, 0.0001, 50)	13	1816.18	990.90	22.03	5	4.31
SC-m-8	5.86	0.05	(1.0, 0.0001, 50)	8	1065.53	922.80	21.57	9	7.88
SC-m-16	4.23	0.05	(1.0, 0.0001, 50)	5	1862.33	1001.98	23.97	16	11.20
PSC	12.85	0.02	(1.0, 0.0001, 50)	4	1809.92	1149.63	10.90	17	12.75
Envelope	6689.78	0.23	(1.0, 0.0001, 50)	3	1949.27	1359.89	11.66	36	20.67
Map-construct	55.43	1.89	(1.0, 0.0001, 50)	1431	817.76	36.36	1454.62	18	5.13

Figure 34: ■ k-means clustering ■ Synthetic ■ 200 initial, 50 trajectories, 50% sampling ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	13.90	0.01	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	13.68	0.02	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-16	14.79	0.03	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-32	22.67	0.05	(1.0, 0.001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-64	405.99	0.09	(1.0, 0.001, 50)	1	353.78	342.26	49.96	12	12.00
SC-m-2	71.68	0.18	(1.0, 0.001, 50)	29	961.82	598.84	44.20	3	1.72
SC-m-4	50.32	0.18	(1.0, 0.001, 50)	25	961.12	602.13	43.09	4	2.00
SC-m-8	62.83	0.18	(1.0, 0.001, 50)	30	961.12	540.95	45.99	8	1.67
SC-m-16	95.14	0.18	(1.0, 0.001, 50)	34	817.76	472.22	48.76	12	1.47
PSC	96.23	0.03	(1.0, 0.001, 50)	13	961.82	549.17	42.58	8	3.23
Envelope	45336.29	0.66	(1.0, 0.001, 50)	7	1603.92	789.47	50.86	13	8.00
Map-construct	178.31	4.57	(1.0, 0.001, 50)	2464	373.52	17.89	2846.05	29	9.57

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	13.43	0.01	(1.0, 0.0003, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	13.81	0.02	(1.0, 0.0003, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-16	27.28	0.03	(1.0, 0.0003, 50)	1	84.89	78.60	49.97	15	15.00
SC-ℓ-32	104.18	0.05	(1.0, 0.0003, 50)	4	352.05	231.16	49.07	24	13.50
SC-ℓ-64	397.95	0.09	(1.0, 0.0003, 50)	4	359.34	343.00	41.02	16	12.00
SC-m-2	58.12	0.18	(1.0, 0.0003, 50)	22	1732.81	816.87	31.44	3	2.27
SC-m-4	37.60	0.18	(1.0, 0.0003, 50)	14	1415.39	789.36	27.67	5	3.57
SC-m-8	28.69	0.18	(1.0, 0.0003, 50)	12	1421.83	862.00	30.38	8	4.58
SC-m-16	31.64	0.18	(1.0, 0.0003, 50)	11	1442.59	794.25	33.62	16	5.36
PSC	46.30	0.03	(1.0, 0.0003, 50)	7	2149.51	909.91	22.25	11	7.14
Envelope	43894.03	0.67	(1.0, 0.0003, 50)	3	1729.07	1127.66	23.34	32	18.67
Map-construct	178.31	4.57	(1.0, 0.0003, 50)	2464	373.52	17.89	2578.74	29	9.57

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	12.76	0.01	(1.0, 0.0001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	12.73	0.02	(1.0, 0.0001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-16	90.47	0.03	(1.0, 0.0001, 50)	9	178.84	141.90	46.91	25	17.89
SC-ℓ-32	100.96	0.05	(1.0, 0.0001, 50)	11	360.63	325.51	40.43	25	17.00
SC-ℓ-64	124.95	0.09	(1.0, 0.0001, 50)	11	721.72	571.37	32.92	19	13.09
SC-m-2	45.39	0.18	(1.0, 0.0001, 50)	19	1419.01	973.50	25.13	3	2.68
SC-m-4	34.44	0.18	(1.0, 0.0001, 50)	13	1422.04	950.39	20.79	5	4.15
SC-m-8	25.84	0.18	(1.0, 0.0001, 50)	9	1426.28	904.15	21.65	9	6.78
SC-m-16	21.04	0.18	(1.0, 0.0001, 50)	6	1512.99	1040.37	22.60	17	11.83
PSC	30.37	0.03	(1.0, 0.0001, 50)	4	1432.23	1195.54	13.14	29	12.50
Envelope	44022.16	0.68	(1.0, 0.0001, 50)	2	2258.96	1496.86	13.82	50	35.50
Map-construct	178.31	4.57	(1.0, 0.0001, 50)	2464	373.52	17.89	2502.36	29	9.57

Figure 35: ■ k-means clustering ■ Synthetic ■ 200 initial, 50 trajectories, 90% sampling ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	13.16	0.01	(1.0, 0.001, 50.0)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	30.28	0.02	(1.0, 0.001, 50.0)	1	0.00	0.00	49.96	22	22.00
SC-ℓ-16	43.91	0.03	(1.0, 0.001, 50.0)	2	47.77	3.11	49.41	17	15.00
SC-ℓ-32	96.20	0.05	(1.0, 0.001, 50.0)	3	92.31	49.53	49.22	12	9.67
SC-ℓ-64	80.36	0.09	(1.0, 0.001, 50.0)	4	240.26	196.18	48.92	13	7.50
SC-m-2	55.69	0.18	(1.0, 0.001, 50.0)	26	903.86	524.29	40.28	3	1.88
SC-m-4	54.11	0.18	(1.0, 0.001, 50.0)	21	929.96	470.10	42.76	4	2.38
SC-m-8	64.99	0.18	(1.0, 0.001, 50.0)	27	483.14	240.02	45.51	9	2.19
SC-m-16	84.95	0.18	(1.0, 0.001, 50.0)	13	903.86	83.37	48.95	17	5.31
PSC	94.45	0.03	(1.0, 0.001, 50.0)	15	877.31	528.49	39.50	6	2.93
Envelope	47045.53	0.71	(1.0, 0.001, 50.0)	8	1777.73	563.85	52.79	16	8.50
Map-construct	234.89	8.29	(1.0, 0.001, 50)	2146	373.52	19.35	2618.79	31	12.29

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	13.26	0.01	(1.0, 0.0003, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	42.82	0.02	(1.0, 0.0003, 50)	2	84.56	23.41	49.67	28	26.50
SC-ℓ-16	93.99	0.03	(1.0, 0.0003, 50)	7	120.80	64.95	47.38	22	18.43
SC-ℓ-32	107.94	0.05	(1.0, 0.0003, 50)	7	237.11	140.27	46.66	21	12.14
SC-ℓ-64	83.44	0.09	(1.0, 0.0003, 50)	8	486.70	390.08	38.49	15	10.88
SC-m-2	48.52	0.18	(1.0, 0.0003, 50)	20	972.17	715.21	28.58	3	2.50
SC-m-4	40.03	0.18	(1.0, 0.0003, 50)	14	966.22	718.40	28.54	5	3.57
SC-m-8	31.05	0.18	(1.0, 0.0003, 50)	11	972.02	654.81	29.57	9	4.55
SC-m-16	28.02	0.18	(1.0, 0.0003, 50)	10	965.05	763.45	33.02	15	5.40
PSC	55.43	0.03	(1.0, 0.0003, 50)	8	1853.11	752.92	22.89	11	6.25
Envelope	47344.87	0.72	(1.0, 0.0003, 50)	3	2274.60	1174.13	21.91	34	16.67
Map-construct	234.89	8.29	(1.0, 0.0003, 50)	2146	373.52	19.35	2287.90	31	12.29

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	11.31	0.01	(1.0, 0.0001, 50)	0	-inf	nan	50.00	-inf	nan
SC-ℓ-8	55.03	0.02	(1.0, 0.0001, 50)	3	84.56	31.98	49.39	28	25.33
SC-ℓ-16	97.14	0.03	(1.0, 0.0001, 50)	9	120.80	70.53	45.70	22	17.11
SC-ℓ-32	206.49	0.05	(1.0, 0.0001, 50)	11	240.31	200.01	42.25	21	12.09
SC-ℓ-64	74.84	0.09	(1.0, 0.0001, 50)	9	967.81	473.16	29.64	24	13.33
SC-m-2	48.07	0.18	(1.0, 0.0001, 50)	19	1924.87	784.42	23.84	3	2.68
SC-m-4	34.69	0.18	(1.0, 0.0001, 50)	13	972.51	744.49	23.28	5	4.08
SC-m-8	25.33	0.18	(1.0, 0.0001, 50)	8	972.02	671.21	23.58	9	7.75
SC-m-16	26.71	0.18	(1.0, 0.0001, 50)	7	1911.65	812.10	25.77	16	11.71
PSC	35.41	0.03	(1.0, 0.0001, 50)	4	1827.31	1109.47	12.41	14	12.75
Envelope	46212.74	0.72	(1.0, 0.0001, 50)	2	2136.43	1445.42	13.93	50	37.00
Map-construct	234.89	8.29	(1.0, 0.0001, 50)	2146	373.52	19.35	2193.36	31	12.29

Figure 36: ■ k-means clustering ■ Synthetic ■ 200 initial, 50 trajectories, 95% sampling ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-m-4	13.18	0.01	(1.0, 0.0003, 100)	4	140.02	63.15	98.42	36	29.75
SC-m-8	10.31	0.01	(1.0, 0.0003, 100)	3	322.77	293.45	98.95	30	23.00
SC-m-16	11.71	0.02	(1.0, 0.0003, 100)	4	648.15	562.78	98.83	23	15.00
SC-m-32	16.44	0.03	(1.0, 0.0003, 100)	17	1295.81	771.20	76.76	17	6.29
SC-m-64	1.42	0.03	(1.0, 0.0003, 100)	0	-inf	nan	100.00	-inf	nan
SC-ℓ-2	25.99	0.03	(1.0, 0.0003, 100)	43	1287.34	961.49	66.47	3	2.35
SC-ℓ-4	16.40	0.03	(1.0, 0.0003, 100)	27	1287.35	904.17	63.92	5	3.70
SC-ℓ-8	10.97	0.03	(1.0, 0.0003, 100)	17	2189.60	980.59	56.42	8	6.06
SC-ℓ-16	8.95	0.03	(1.0, 0.0003, 100)	17	1284.38	977.10	59.43	14	5.88
PSC	17.53	0.02	(1.0, 0.0003, 100)	9	1519.86	1071.38	39.91	27	11.11
Envelope	2118.46	0.28	(1.0, 0.0003, 100)	3	2068.23	1307.59	51.86	56	34.67
Map-construct	64.70	2.62	(1.0, 0.0003, 100)	1629	579.49	44.43	1748.39	37	6.40

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-m-4	27.76	0.01	(1.0, 0.0001, 100)	13	311.99	146.19	89.46	100	38.15
SC-m-8	22.67	0.01	(1.0, 0.0001, 100)	15	646.15	364.20	83.79	48	26.13
SC-m-16	10.92	0.02	(1.0, 0.0001, 100)	15	1287.34	677.37	58.73	47	20.53
SC-m-32	9.23	0.03	(1.0, 0.0001, 100)	8	1296.25	1044.93	54.22	23	18.25
SC-m-64	1.59	0.03	(1.0, 0.0001, 100)	0	-inf	nan	100.00	-inf	nan
SC-ℓ-2	22.44	0.03	(1.0, 0.0001, 100)	39	1289.06	1045.58	54.15	3	2.64
SC-ℓ-4	14.66	0.03	(1.0, 0.0001, 100)	23	1291.72	1066.94	44.45	5	4.70
SC-ℓ-8	9.09	0.03	(1.0, 0.0001, 100)	18	2583.02	1081.79	39.06	9	7.00
SC-ℓ-16	5.91	0.03	(1.0, 0.0001, 100)	11	1570.96	1080.64	36.68	16	10.27
PSC	12.35	0.02	(1.0, 0.0001, 100)	5	1287.25	1116.28	18.12	37	20.00
Envelope	2085.27	0.28	(1.0, 0.0001, 100)	3	2197.06	1490.34	24.44	72	41.67
Map-construct	64.70	2.62	(1.0, 0.0001, 100)	1629	579.49	44.43	1669.45	37	6.40

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-m-4	18.81	0.01	(1.0, 0.00003, 100)	20	1284.38	381.59	66.48	337	111.40
SC-m-8	16.92	0.01	(1.0, 0.00003, 100)	22	1295.81	603.50	57.61	103	43.36
SC-m-16	8.60	0.02	(1.0, 0.00003, 100)	11	1295.79	885.25	48.74	87	41.27
SC-m-32	8.26	0.03	(1.0, 0.00003, 100)	6	1295.81	1056.94	42.22	46	29.17
SC-m-64	1.58	0.03	(1.0, 0.00003, 100)	0	-inf	nan	100.00	-inf	nan
SC-ℓ-2	20.28	0.03	(1.0, 0.00003, 100)	36	1289.06	1126.99	46.84	3	2.89
SC-ℓ-4	13.52	0.03	(1.0, 0.00003, 100)	23	1295.81	1151.16	36.86	5	4.70
SC-ℓ-8	8.27	0.03	(1.0, 0.00003, 100)	14	1295.81	1176.01	28.03	9	8.71
SC-ℓ-16	5.40	0.03	(1.0, 0.00003, 100)	10	2556.53	1187.21	23.70	17	13.30
PSC	10.29	0.02	(1.0, 0.00003, 100)	5	2484.46	1246.04	12.94	61	36.20
Envelope	2093.53	0.28	(1.0, 0.00003, 100)	2	2506.83	1524.09	9.57	100	71.00
Map-construct	64.70	2.62	(1.0, 0.00003, 100)	1629	579.49	44.43	1641.82	37	6.40

Figure 37: ■ k-means clustering ■ Synthetic ■ 100 initial, 100 trajectories, 50% sampling ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	147.32	0.01	(1.0, 0.0003, 100)	29	140.02	14.49	83.53	53	31.90
SC-ℓ-8	99.36	0.02	(1.0, 0.0003, 100)	17	148.12	40.86	84.10	36	23.71
SC-ℓ-16	86.28	0.03	(1.0, 0.0003, 100)	12	148.12	93.18	88.42	24	12.08
SC-ℓ-32	73.77	0.05	(1.0, 0.0003, 100)	9	603.47	323.66	84.64	22	10.78
SC-ℓ-64	51.23	0.09	(1.0, 0.0003, 100)	12	1206.42	693.63	63.72	14	7.92
SC-m-2	88.70	0.09	(1.0, 0.0003, 100)	46	1197.86	703.87	64.87	3	2.17
SC-m-4	52.66	0.09	(1.0, 0.0003, 100)	25	1196.71	720.27	55.00	5	4.00
SC-m-8	34.67	0.09	(1.0, 0.0003, 100)	16	1207.70	643.86	57.41	9	6.88
SC-m-16	40.68	0.09	(1.0, 0.0003, 100)	16	1184.38	366.98	70.68	17	12.88
PSC	120.80	0.03	(1.0, 0.0003, 100)	15	1207.70	651.92	52.80	19	8.07
Envelope	15357.74	0.76	(1.0, 0.0003, 100)	5	1932.07	1287.87	48.31	38	20.40
Map-construct	278.55	11.79	(1.0, 0.0003, 100)	3015	421.42	22.83	3282.58	55	13.89

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	145.69	0.01	(1.0, 0.0001, 100)	32	146.82	33.03	76.10	53	35.25
SC-ℓ-8	91.08	0.02	(1.0, 0.0001, 100)	18	602.06	110.84	77.02	46	26.61
SC-ℓ-16	75.86	0.03	(1.0, 0.0001, 100)	17	603.48	353.49	74.35	41	21.76
SC-ℓ-32	55.62	0.05	(1.0, 0.0001, 100)	12	1204.63	596.68	59.99	38	20.08
SC-ℓ-64	44.74	0.09	(1.0, 0.0001, 100)	9	1205.95	841.22	49.33	22	12.11
SC-m-2	76.09	0.09	(1.0, 0.0001, 100)	40	1206.42	938.99	51.99	3	2.52
SC-m-4	49.05	0.09	(1.0, 0.0001, 100)	26	1205.95	764.59	41.97	5	4.04
SC-m-8	30.29	0.09	(1.0, 0.0001, 100)	16	1207.70	877.98	38.04	9	8.31
SC-m-16	22.97	0.09	(1.0, 0.0001, 100)	11	1207.70	925.68	39.97	17	13.91
PSC	41.06	0.03	(1.0, 0.0001, 100)	6	2249.71	1170.41	22.05	29	17.67
Envelope	15947.31	0.78	(1.0, 0.0001, 100)	3	2671.49	1408.18	27.03	100	50.00
Map-construct	278.55	11.79	(1.0, 0.0001, 100)	3015	421.42	22.83	3104.24	55	13.89

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	121.60	0.01	(1.0, 0.00003, 100)	31	295.08	108.12	66.87	333	53.23
SC-ℓ-8	76.29	0.02	(1.0, 0.00003, 100)	18	602.07	205.03	70.55	109	35.33
SC-ℓ-16	58.30	0.03	(1.0, 0.00003, 100)	21	1204.43	522.34	51.47	89	30.19
SC-ℓ-32	43.62	0.05	(1.0, 0.00003, 100)	12	2399.86	818.69	41.84	65	28.50
SC-ℓ-64	20.58	0.09	(1.0, 0.00003, 100)	4	1205.95	1157.54	40.92	53	33.00
SC-m-2	70.71	0.09	(1.0, 0.00003, 100)	39	1815.92	946.64	47.33	3	2.59
SC-m-4	47.07	0.09	(1.0, 0.00003, 100)	25	1811.34	928.02	35.48	5	4.40
SC-m-8	29.31	0.09	(1.0, 0.00003, 100)	15	1205.75	897.35	28.27	9	8.60
SC-m-16	19.64	0.09	(1.0, 0.00003, 100)	9	1205.34	1121.15	25.70	17	16.11
PSC	25.54	0.03	(1.0, 0.00003, 100)	2	2322.02	1906.28	7.68	76	50.50
Envelope	15598.71	0.78	(1.0, 0.00003, 100)	3	2758.78	1492.12	11.34	90	58.67
Map-construct	278.55	11.79	(1.0, 0.00003, 100)	3015	421.42	22.83	3041.83	55	13.89

Figure 38: ■ k-means clustering ■ Synthetic ■ 100 initial, 100 trajectories, 90% sampling ■

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	195.85	0.01	(1.0, 0.0003, 100)	30	98.66	5.32	75.38	52	36.37
SC-ℓ-8	141.00	0.02	(1.0, 0.0003, 100)	18	204.52	19.99	72.08	40	29.39
SC-ℓ-16	96.05	0.03	(1.0, 0.0003, 100)	13	216.45	45.86	71.05	33	20.23
SC-ℓ-32	144.36	0.05	(1.0, 0.0003, 100)	20	444.13	219.70	72.34	21	8.75
SC-ℓ-64	118.57	0.09	(1.0, 0.0003, 100)	15	877.85	559.71	63.90	16	7.13
SC-m-2	118.84	0.09	(1.0, 0.0003, 100)	45	1762.04	626.53	60.57	3	2.22
SC-m-4	72.66	0.09	(1.0, 0.0003, 100)	25	1743.80	621.66	51.04	5	4.04
SC-m-8	49.98	0.09	(1.0, 0.0003, 100)	19	883.99	380.70	55.85	9	7.26
SC-m-16	75.48	0.09	(1.0, 0.0003, 100)	21	880.75	52.24	67.16	17	15.95
PSC	194.50	0.03	(1.0, 0.0003, 100)	19	877.85	142.52	56.43	28	14.53
Envelope	17665.14	0.82	(1.0, 0.0003, 100)	9	1892.40	663.55	51.14	33	19.67
Map-construct	403.95	11.74	(1.0, 0.0003, 100)	2513	290.61	17.40	2755.09	53	19.39

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	184.59	0.01	(1.0, 0.0001, 100)	32	108.56	14.42	71.24	54	38.03
SC-ℓ-8	123.70	0.02	(1.0, 0.0001, 100)	19	218.56	52.31	63.52	44	33.32
SC-ℓ-16	100.55	0.03	(1.0, 0.0001, 100)	19	442.68	172.57	64.26	43	21.21
SC-ℓ-32	118.62	0.05	(1.0, 0.0001, 100)	19	887.61	504.44	58.10	34	14.79
SC-ℓ-64	84.02	0.09	(1.0, 0.0001, 100)	11	1767.41	823.93	48.34	26	12.18
SC-m-2	113.72	0.09	(1.0, 0.0001, 100)	44	1762.04	801.84	52.57	3	2.30
SC-m-4	66.26	0.09	(1.0, 0.0001, 100)	25	1762.75	769.69	39.04	5	4.12
SC-m-8	42.42	0.09	(1.0, 0.0001, 100)	18	1746.32	674.98	40.75	9	7.78
SC-m-16	28.62	0.09	(1.0, 0.0001, 100)	12	1769.23	923.29	37.90	17	13.17
PSC	60.53	0.03	(1.0, 0.0001, 100)	7	1762.04	911.87	23.59	29	19.71
Envelope	17668.48	0.83	(1.0, 0.0001, 100)	5	2216.57	1274.68	27.97	70	34.40
Map-construct	403.95	11.74	(1.0, 0.0001, 100)	2513	290.61	17.40	2593.74	53	19.39

Name	Seconds	GBytes	c	Clusters	Max Frechet	Avg Frechet	kMeans	Max Size	Avg Size
SC-ℓ-4	149.65	0.01	(1.0, 0.00003, 100)	32	216.45	88.12	62.72	229	51.19
SC-ℓ-8	112.32	0.02	(1.0, 0.00003, 100)	19	442.25	139.13	58.67	56	38.37
SC-ℓ-16	74.14	0.03	(1.0, 0.00003, 100)	20	1751.21	525.14	48.86	130	36.55
SC-ℓ-32	84.72	0.05	(1.0, 0.00003, 100)	12	1760.92	665.62	46.57	75	27.92
SC-ℓ-64	95.36	0.09	(1.0, 0.00003, 100)	5	1762.75	955.08	42.88	38	27.60
SC-m-2	93.59	0.09	(1.0, 0.00003, 100)	39	2101.30	885.17	45.10	3	2.59
SC-m-4	64.59	0.09	(1.0, 0.00003, 100)	25	1762.75	864.45	35.30	5	4.44
SC-m-8	38.23	0.09	(1.0, 0.00003, 100)	14	1764.76	1028.71	26.39	9	8.36
SC-m-16	27.22	0.09	(1.0, 0.00003, 100)	10	1776.04	1130.93	25.60	17	14.40
PSC	24.24	0.03	(1.0, 0.00003, 100)	1	1766.90	1704.32	7.32	100	100.00
Envelope	17665.40	0.84	(1.0, 0.00003, 100)	3	2492.02	1486.45	11.39	100	57.00
Map-construct	403.95	11.74	(1.0, 0.00003, 100)	2513	290.61	17.40	2537.27	53	19.39

Figure 39: ■ k-means clustering ■ Synthetic ■ 100 initial, 100 trajectories, 95% sampling ■

References

[1] Anonymous repository. https://anonymous.4open.science/r/Subtrajectory-Clustering-Test-Suite-C13F/.

[2] MoveTK: the movement toolkit. http://https://github.com/movetk/movetk, 2020.

[3] Pankaj K Agarwal, Kyle Fox, Kamesh Munagala, Abhinandan Nath, Jiangwei Pan, and Erin Taylor. Subtrajectory clustering: Models and algorithms. ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS), pages 75–87, 2018. doi:10.1145/3196959.3196972.

[4] Mahmuda Ahmed, Sophia Karagiorgou, Dieter Pfoser, and Carola Wenk. A comparison and evaluation of map construction algorithms using vehicle tracking data. GeoInformatica, 19:601–632, 2015. doi:10.1007/s10707-014-0222-6.

[5] Mahmuda Ahmed and Carola Wenk. Constructing street networks from gps trajectories. European Symposium on Algorithms (ESA), pages 60–71, 2012. doi:10.1007/978-3-642-33090-2_7.

[6] Hugo Akitaya, Frederik Brüning, Erin Chambers, and Anne Driemel. Subtrajectory Clustering: Finding Set Covers for Set Systems of Subcurves. Computing in Geometry and Topology, 2(1):1:1–1:48, 2023. doi:10.57717/cgt.v2i1.7.

[7] Helmut Alt and Michael Godau. Computing the fréchet distance between two polygonal curves. International Journal of Computational Geometry & Applications, 5:75–91, 1995. doi:10.1142/S0218195995000064.

[8] James Biagioni and Jakob Eriksson. Map inference in the face of noise and disparity. ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL), pages 79–88, 2012. doi:10.1145/2424321.2424333.

[9] Jiang Bian, Dayong Tian, Yuanyan Tang, and Dacheng Tao. A survey on trajectory clustering analysis. arXiv preprint arXiv:1802.06971, 2018.

[10] Frederik Brüning, Jacobus Conradi, and Anne Driemel. Faster Approximate Covering of Subcurves Under the Fréchet Distance. In Shiri Chechik, Gonzalo Navarro, Eva Rotenberg, and Grzegorz Herman, editors, European Symposium on Algorithms (ESA 2022), volume 244 of Leibniz International Proceedings in Informatics (LIPIcs), pages 28:1–28:16, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ESA.2022.28.

[11] Kevin Buchin, Maike Buchin, David Duran, Brittany Terese Fasy, Roel Jacobs, Vera Sacristan, Rodrigo I Silveira, Frank Staals, and Carola Wenk. Clustering trajectories for map construction. ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL), pages 1–10, 2017. doi:10.1145/3139958.3139964.

[12] Kevin Buchin, Maike Buchin, Joachim Gudmundsson, Jorren Hendriks, Erfan Hosseini Sereshgi, Vera Sacristán, Rodrigo I Silveira, Jorrick Sleijster, Frank Staals, and Carola Wenk. Improved map construction using subtrajectory clustering. ACM Workshop on Location-Based Recommendations, Geosocial Networks, and Geoadvertising (SIGSPATIAL), pages 1–4, 2020. doi:10.1145/3423334.3431451.

[13] Kevin Buchin, Maike Buchin, Joachim Gudmundsson, Jorren Hendriks, Erfan Hosseini Sereshgi, Rodrigo I Silveira, Jorrick Sleijster, Frank Staals, and Carola Wenk. Roadster: Improved algorithms for subtrajectory clustering and map construction. Computers & Geosciences, 196:105845, 2025. doi:10.1016/j.cageo.2024.105845.

[14] Kevin Buchin, Maike Buchin, Joachim Gudmundsson, Maarten Löffler, and Jun Luo. Detecting commuting patterns by clustering subtrajectories. International Journal of Computational Geometry & Applications, 21(03):253–282, 2011. doi:10.1142/S0218195911003638.

[15] Kevin Buchin, Maike Buchin, Marc Van Kreveld, Maarten Löffler, Rodrigo I Silveira, Carola Wenk, and Lionov Wiratma. Median trajectories. Algorithmica, 66:595–614, 2013. doi:10.1007/s00453-012-9654-2.

[16] Lili Cao and John Krumm. From gps traces to a routable road map. ACM international conference on advances in geographic information systems (SIGSPATIAL), pages 3–12, 2009. doi:10.1145/1653771.1653776.

[17] Chen Chen, Hao Su, Qixing Huang, Lin Zhang, and Leonidas Guibas. Pathlet learning for compressing and planning trajectories. ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL), pages 392–395, 2013. doi:10.1145/2525314.2525443.

[18] Jacobus Conradi and Anne Driemel. Finding complex patterns in trajectory data via geometric set cover. arXiv preprint arXiv:2308.14865, 2023.

[19] Jonathan J Davies, Alastair R Beresford, and Andy Hopper. Scalable, distributed, real-time map generation. IEEE Pervasive Computing, 5(4):47–54, 2006. doi:10.1109/MPRV.2006.83.

[20] Mark de Berg, Otfried Cheong, Marc J. van Kreveld, and Mark H. Overmars. Computational geometry: algorithms and applications, 3rd Edition. Springer, 2008.

[21] Scott Gaffney and Padhraic Smyth. Trajectory clustering with mixtures of regression models. ACM International Conference on Knowledge Discovery and Data mining (SIGKDD), pages 63–72, 1999. doi:10.1145/312129.312197.

[22] Joachim Gudmundsson, Andreas Thom, and Jan Vahrenhold. Of motifs and goals: mining trajectory data. ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL), pages 129–138, 2012. doi:10.1145/2424321.2424339.

[23] Joachim Gudmundsson and Nacho Valladares. A GPU approach to subtrajectory clustering using the fréchet distance. ACM nternational Conference on Advances in Geographic Information Systems (SIGSPATIAL), pages 259–268, 2012. doi:10.1145/2424321.2424355.

[24] Joachim Gudmundsson and Sampson Wong. Cubic upper and lower bounds for subtrajectory clustering under the continuous fréchet distance. ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 173–189, 2022. doi:10.1137/1.9781611977073.9.

[25] Jincai Huang, Min Deng, Jianbo Tang, Shuling Hu, Huimin Liu, Sembeto Wariyo, and Jinqiang He. Automatic generation of road maps from low quality gps trajectory data via structure learning. IEEE Access, 6:71965–71975, 2018. doi:10.1109/ACCESS.2018.2882581.

[26] Chih-Chieh Hung, Wen-Chih Peng, and Wang-Chien Lee. Clustering and aggregating clues of trajectories for mining trajectory patterns and routes. The VLDB Journal, 24:169–192, 2015. doi:10.1007/s00778-011-0262-6.

[27] Sophia Karagiorgou and Dieter Pfoser. On vehicle tracking data-based road network generation. ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL), pages 89–98, 2012. doi:10.1145/2424321.2424334.

[28] Sophia Karagiorgou, Dieter Pfoser, and Dimitrios Skoutas. Segmentation-based road network construction. ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL), pages 460–463, 2013. doi:10.1145/2525314.2525460.

[29] Jae-Gil Lee, Jiawei Han, and Kyu-Young Whang. Trajectory clustering: a partition-and-group framework. ACM International Conference on Management of Data (SIGMOD), pages 593–604, 2007. doi:10.1145/1247480.1247546.

[30] Vanessa Lago Machado, Ronaldo dos Santos Mello, Vânia Bogorny, and Geomar André Schreiner. A survey on the computation of representative trajectories. GeoInformatica, pages 1–26, 2024. doi:10.1007/s10707-024-00514-y.

[31] Tobias Moers, Lennart Vater, Robert Krajewski, Julian Bock, Adrian Zlocki, and Lutz Eckstein. The exid dataset: A real-world trajectory dataset of highly interactive highway sce-

narios in germany. 2022 IEEE Intelligent Vehicles Symposium (IV), pages 958–964, 2022. doi:10.1109/IV51971.2022.9827305.

[32] Cynthia Sung, Dan Feldman, and Daniela Rus. Trajectory clustering for motion prediction. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 1547–1552, 2012. doi:10.1109/IROS.2012.6386017.

[33] Suyi Wang, Yusu Wang, and Yanjie Li. Efficient map reconstruction and augmentation via topological methods. ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL), pages 1–10, 2015. doi:10.1145/2820783.2820833.